USERS

Vol. 1



React Native

Entorno de trabajo
Estilos y contenedores
Estructura de datos

React Native - Vol.1

Entorno de trabajo - Estilos y contenedores - Estructura de datos



Título: React Native - Vol. 01

Entorno de trabajo - Estilos y contenedores - Estructura de datos

Autor: Santiago Aguirre / **Colección**: USERS ebooks - LPCU 409

Prólogo

React Native es un nuevo framework para el desarrollo de aplicaciones nativas en dispositivos móviles, que permite trabajar con el lenguaje JavaScript utilizando las bases de la popular librería React. Desde su aparición, el mercado de aplicaciones para teléfonos ha crecido de manera exponencial, incluso inspirado en el surgimiento de otras tecnologías similares que facilitan y acercan a muchos al complejo mundo del desarrollo para dispositivos móviles. Si estás interesado en comenzar a crear tus propias aplicaciones y adentrarte en el mundo del desarrollo móvil, que ha tenido una demanda enorme en el último tiempo, esta entrega es la forma perfecta para comenzar a trabajar.

Sobre esta colección

En esta colección estudiarás todos los aspectos iniciales del desarrollo en React Native. Comenzarás a crear tu propia aplicación con el framework basado en React desarrollado por Meta, anteriormente Facebook, y que logró abarcar una gran cuota de mercado en el mundo de la programación. Conocerás Expo, una plataforma de código abierto que te ayudará a emular, desarrollar y distribuir aplicaciones para dispositivos móviles.

ESTE E-BOOK INCLUYE

- **ACTIVIDADES**: al final de cada capítulo, una serie de preguntas para que puedas autoevaluarte y ejercicios prácticos.

- **MATERIAL ADICIONAL**: cada vez que veas la señal de la derecha, significará que encontrarás material disponible en el archivo de contenidos complementarios que puedes descargar AQUÍ.



Verás cómo crear desde cero una aplicación con Expo utilizando la línea de comandos y de qué manera personalizarla como desees.
También aprenderás a trabajar con distintos aspectos del framework y con los componentes más importantes que ofrece React Native para manejarte con texto, listas, imágenes, y moverte entre las diferentes pantallas de una aplicación. Verás cómo trabajar con los estilos de una aplicación, dándole a cada componente características distintas que sirvan a los propósitos que quieras lograr. También verás la forma de utilizar datos externos desde un servicio web alojado en un servidor, que te permita administrar los datos, y que puedan ser enviados a todas las instancias instaladas de la aplicación. Podrás realizar peticiones HTTP y aprenderás a mostrarlas en pantalla como una lista de datos. Por último, verás cómo enviar información con peticiones de regreso al servidor.



Volumen I

Aprenderás sobre el mundo de las aplicaciones móviles y la ventaja de trabajar con una tecnología única para desarrollar en dispositivos tanto con iOS como con Android. Verás cómo crear tus propios componentes y cómo trabajar con los que provee React Native.



Volumen II

Avanzarás en la creación de una aplicación de React Native con aspectos avanzados como autenticación, protección de pantallas con un sistema de inicio de sesión y almacenamiento de datos.

Contenido

CAPÍTULO 1 React Native

Introducción / 9

Android e iOS / 10

Entorno de trabajo / 18

Comenzar a desarrollar / 24

Actividades / 33

- Test de autoevaluación / 33
- Ejercicios prácticos / 33

CAPÍTULO 2 Estilos

Estilos y contenedores / 35

Navegación y pantallas / 44

Actividades / 58

- Test de autoevaluación / 58
- Ejercicios prácticos / 58

CAPÍTULO 3 Datos

Estructuras de datos / 60

Listas y componentes nativos / 66

Peticiones y objetos anidados / 77

Actividades / 82

- Test de autoevaluación / 82
- Ejercicios prácticos / 82

CAPÍTULO 4 API externa

Trabajar con los datos del servidor / 84

Formularios en React Native / 94

Actividades / 109

- Test de autoevaluación / 109
- Ejercicios prácticos / 109

Glosario / 110

Capítulo 01



React Native

En los últimos años, el desarrollo de software y, en particular, el de aplicaciones nativas para móviles, ha crecido tanto en comunidad como en tecnologías disponibles para el programador, lo que permite acceder a nuevas características y trabajar de manera más ágil, cómoda y escalable.

Introducción / 9

Android e iOS / 10

Entorno de trabajo / 18

Comenzar a desarrollar / 24

Actividades / 33

- Test de autoevaluación / 33
- Ejercicios prácticos / 33

Introducción

Cuando hablamos de aplicaciones móviles, nos referimos, particularmente, a aquellas instalables en un dispositivo portátil, como un smartphone o una tablet, dejando de lado las computadoras personales de tipo laptops portátiles y de escritorio. En los últimos años, los dispositivos móviles han ido creciendo tanto en producción como en poder de cómputo, y hoy existen teléfonos con varios núcleos, memoria RAM muchísimo mayor, gran capacidad de almacenamiento y mejoras en sus pantallas, con lo cual su popularidad llega al punto de que la mayoría de las personas llevan un teléfono móvil consigo todo el tiempo. Es importante aclarar que, cuando hablamos de dispositivos móviles, smartphones o teléfonos, nos estamos refiriendo al mismo tipo de dispositivo.

Con la popularización de estos equipos, el alcance de la Web ha ido creciendo; nos referimos al acceso a páginas no solo mediante el navegador, sino también a través de otros programas instalables en el sistema operativo de los smartphones, tal como se instalan programas en una computadora, como un lector de archivos PDF, un visor de videos e imágenes, un reproductor de música y software de geolocalización, entre muchísimos otros ejemplos. Algunos de estos programas, como todo sistema operativo, ya están instalados por defecto en el equipo, mientras que otros deben ser instalados por el usuario.

Hace años, antes de que los teléfonos móviles se volvieran populares, el desarrollo de software se centraba en la creación de páginas web sencillas o de programas instalables en computadoras personales, que podían adquirirse en un local físico, comprando un disco compacto, o mediante una página web. A medida que los teléfonos fueron cobrando más protagonismo en la vida diaria de las personas, y con la mejora de las redes en muchos países, quedó claro que un gran mercado del desarrollo web debía satisfacer las demandas de estos dispositivos.

Con la creación del iPhone primero y con otras marcas ofreciendo dispositivos más personalizables, el desarrollo para aplicaciones móviles fue tomando forma.

Android e iOS

Para entender la existencia de las stores, o mercados de aplicaciones, en los teléfonos, es necesario comprender un poco de la historia de los smartphones y de dónde nacen los sistemas operativos.
Linux es conocido por ser el sistema operativo open source más utilizado en aplicaciones comerciales. Si bien ocupa una cuota de mercado muy baja para computadoras personales, la mayoría de los servidores hoy corren este sistema, y con su popularización, también se fue aprovechado para la creación de uno de los sistemas más usados por dispositivos móviles a nivel global. Nos referimos a Android, que, aunque muchísima gente no lo sabe, está basado en el sistema operativo creado por Linus Torvalds, principalmente en su kernel o núcleo.
El kernel es una base fundamental de todo sistema operativo, que permite acceder de forma segura a los recursos del hardware. Android en particular está basado en este **`kernel`** creado por Linux, también de código abierto y que, además de correr en teléfonos móviles, es utilizado ampliamente en todo tipo de dispositivos, ya que no necesita una gran cantidad de recursos de hardware, y permite configurar múltiples características de un aparato. Se lo emplea en tablets, estéreos de automóviles, relojes inteligentes, televisores y consolas de videojuegos (tanto de sobremesa como portátiles); incluso, la conocida empresa **`Nvidia`** creó su propia consola de videojuegos portátiles con un sistema operativo corriendo sobre Android (Figura 1.1.).

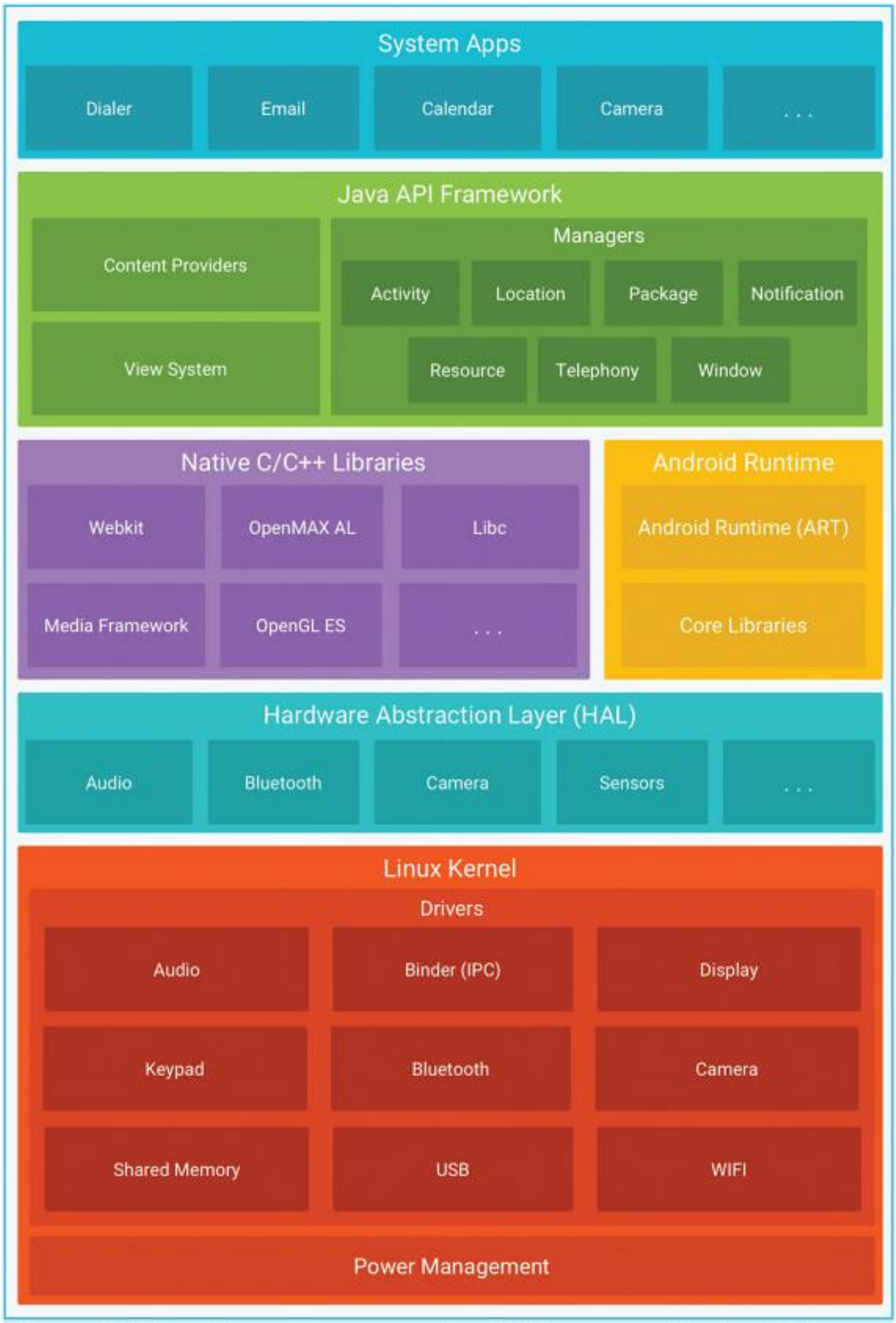


Figura 1.1. *En este esquema se muestra el sistema operativo Android con sus principales características.*

La versatilidad del sistema operativo permite que incluso computadoras portátiles corran con Android, y dado que es de código abierto, muchas empresas optan por utilizar la licencia y el código de este sistema, y crear sobre él su capa de personalización, cambiando sus características más importantes según las necesidades del dispositivo. En el caso de los televisores, el sistema es adaptado para poder sintonizar emisoras utilizando su hardware. En los GPS que utilizan Android, el sistema se personaliza para mostrar mapas y actualizarse constantemente con Internet, así como también para mostrar en todo momento la geolocalización por medio de satélites.

En los teléfonos portátiles, el sistema operativo es adquirido bajo licencia de la empresa Google, que mantiene y desarrolla Android, y se personaliza dependiendo de sus características. Por ejemplo, Samsung, con sus teléfonos Galaxy, se encarga de comprar la licencia y luego personalizar la capa en sus equipo, dándoles una interfaz de usuario con características propias de la marca, así como también aplicando cambios dependiendo del hardware, adaptado a sus cámaras, a su almacenamiento y otras características.

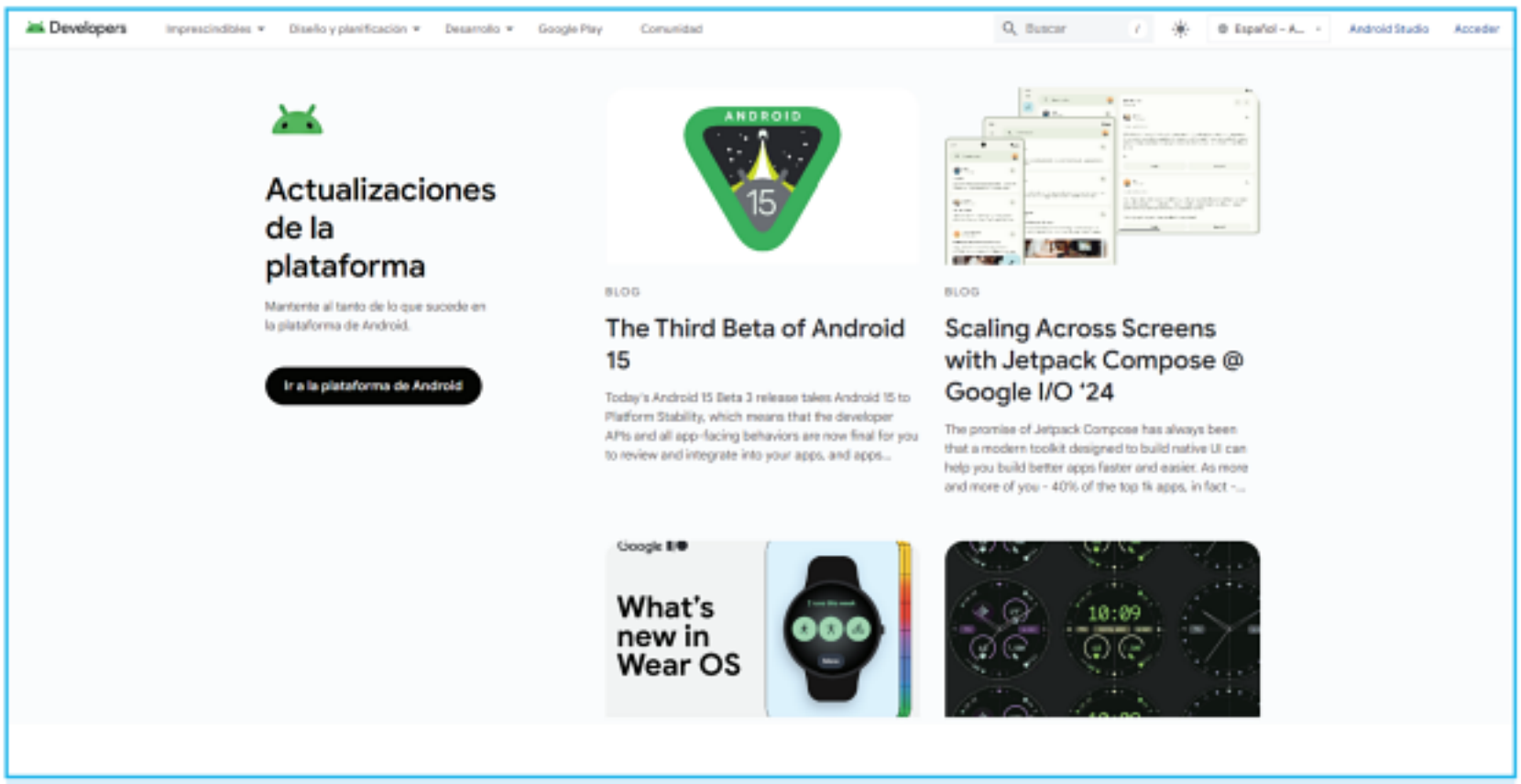


Figura 1.2. *Android es mantenido y desarrollado por la empresa Google.*

Android corre con aplicaciones que se pueden instalar mediante archivos de tipo **APK**, disponibles en el mercado de aplicaciones oficial, que de ahora en más llamaremos Google Play Store, o descargando el archivo **APK**. Estas aplicaciones, en principio, pueden desarrollarse en el lenguaje de programación Java, que es, además, uno de los utilizados para el sistema operativo. En su defecto, pueden hacerse en el lenguaje creado por Google Kotlin, muy parecido a **Java**, también de tipado estricto y de alto nivel, que incluso puede ejecutarse una vez compilado por una JVM compatible.

El otro ecosistema más popular a nivel global es iOS, un sistema operativo para dispositivos móviles creado por Apple que, a diferencia de Android, es propietario o privativo, no es de código abierto y corre sobre dispositivos desarrollados por la misma compañía. Esto significa que no es posible obtener el código del sistema, modificarlo a gusto e instalarlo en un dispositivo propio, como sí puede hacerse con Android.

A diferencia de este, iOS tiene su propio mercado de aplicaciones, la App Store, y corre con programas creados con el lenguaje de programación Objective C o, en su defecto, con el lenguaje desarrollado por Apple, **`Swift`**.

Trabajando en una computadora Mac, estos lenguajes permiten crear aplicaciones para teléfonos con el sistema operativo iOS, que se publican luego de un proceso de validación, aunque también pueden distribuirse con el sistema **`Test Flight`**, para lanzamientos internos, de modo que amigos o familiares pueden probarla.

Publicar una aplicación en los mercados de ambos ecosistemas no es gratuito, sino que requiere el pago de una cuota anual en el caso de Apple, o un pago único para Android (Figura 1.3.).

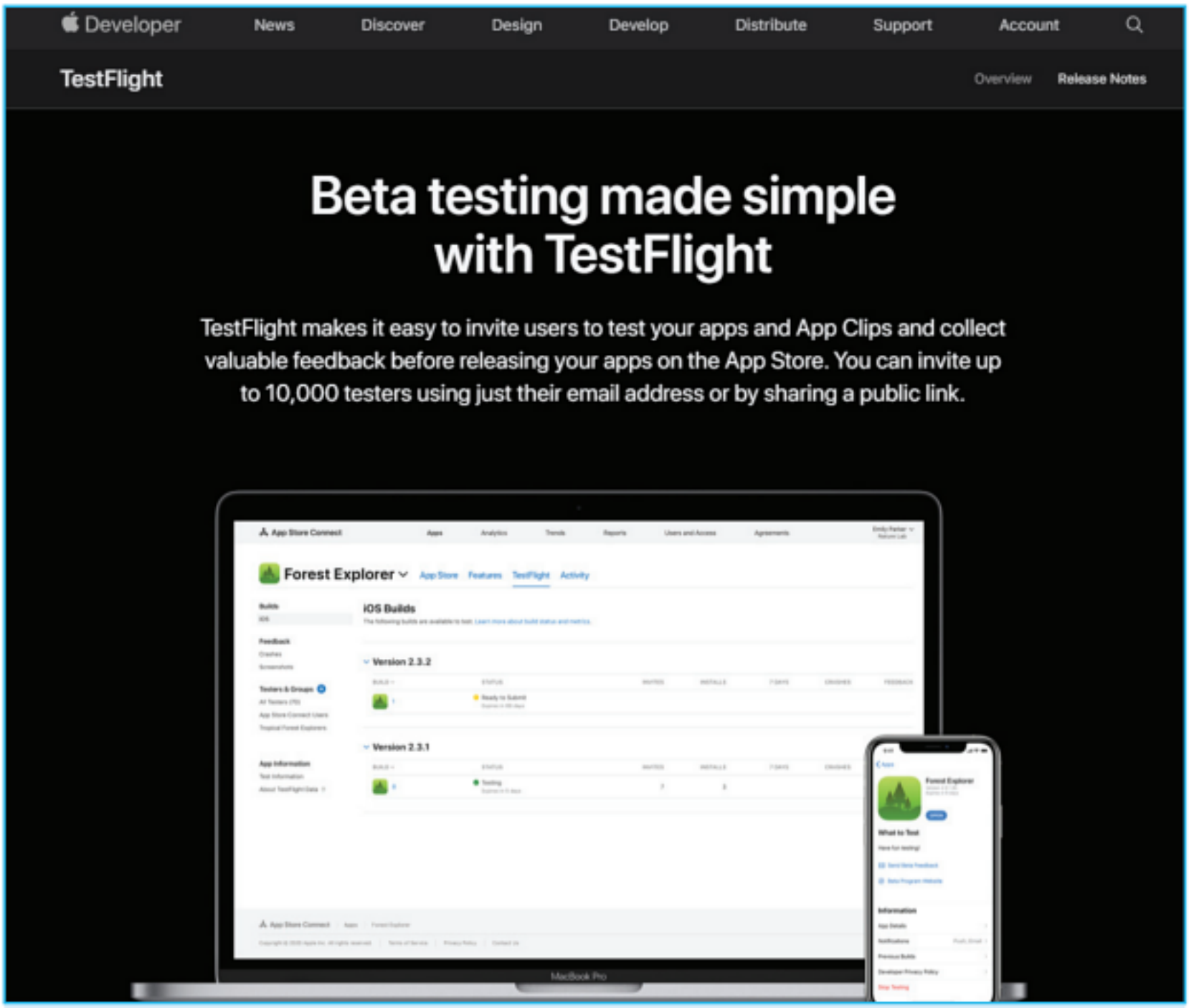


Figura 1.3. *Apple ofrece el software Test Flight para lanzamientos internos y pruebas sobre aplicaciones.*

El problema con ambos sistemas es claro: ninguno de los dos permite instalar una aplicación desarrollada para el otro. Es decir, si desarrollas una aplicación en **Java** o en **Kotlin** para Android, no podría instalarse en un dispositivo de Apple con iOS. Esto representa un problema, dado que el desarrollador o equipo de desarrollo se ve obligado, durante años, a mantener dos programas idénticos para dos plataformas distintas.
A diferencia de un programa creado en Java, que corre sobre una **JVM** instalable en Windows, Linux o MacOS, un dispositivo de iOS no podría correr un **APK** ya compilado. Durante años, esto fue un problema dado que se requería un equipo de desarrollo que supiera manejar ambos lenguajes

para generar una única aplicación, situación que resulta en mayores costos para el proyecto y tiempos más prolongados para crearla.
Una solución a este problema llegó hace unos años con el nacimiento de ciertas tecnologías multiplataforma basadas en un lenguaje con una curva de aprendizaje menor y accesible para todos: **JavaScript**. El lenguaje utilizado para la programación de la Web fue rápidamente aceptado por desarrolladores, ya que no programación mucho tiempo aprenderlo y es sencillo de utilizar. Permite trabajar con clases y objetos, aunque esto no es completamente obligatorio y puede correr en cualquier dispositivo. Por esta razón, han comenzado a aparecer nuevos entornos de trabajo, o frameworks, que facilitan el desarrollo de programas y, aún más, el aprendizaje. Basados en diversas plataformas, como Ionic sobre Angular o, incluso, con nuevos lenguajes como **Dart**, para el framework **Flutter**, ayudan a construir aplicaciones multiplataforma.

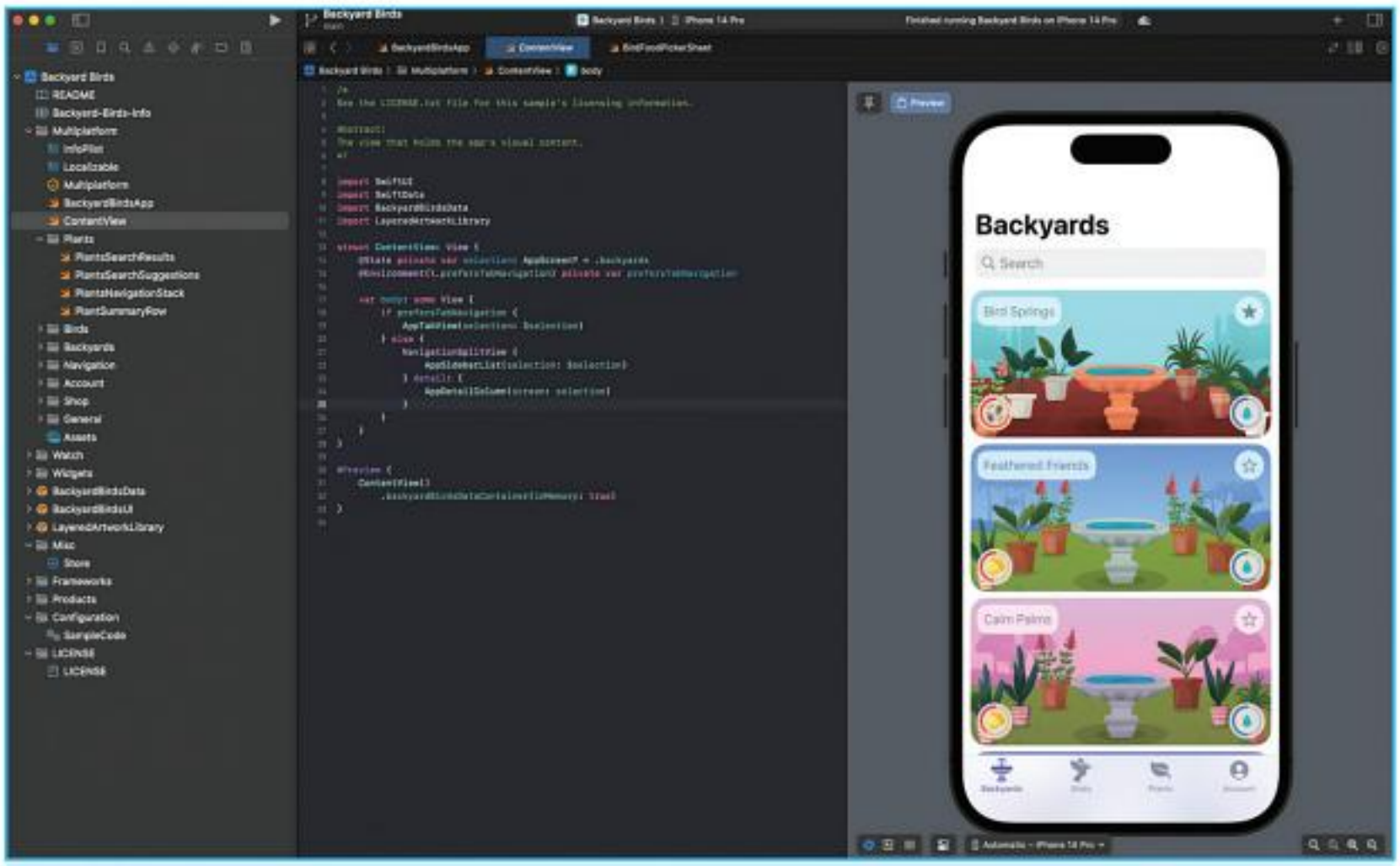


Figura 1.4. *Para el desarrollo de aplicaciones es indispensable contar con un emulador de dispositivos.*

El caso de **React** es muy claro, ya que el desarrollo de aplicaciones web ha crecido muchísimo con el uso de esta librería creada por la empresa Meta, en ese momento Facebook. Permite generar aplicaciones del lado del cliente de forma mucho más rápida que con **JavaScript**, **CSS** o **HTML** nativo. En paralelo, algún tiempo después y respondiendo a la necesidad de facilitar el desarrollo para aplicaciones móviles, nació React Native, un framework creado y mantenido también por Meta para desarrollar aplicaciones móviles en un único lenguaje y plataforma, que corran en distintos dispositivos móviles o que utilicen los diferentes sistemas operativos portátiles, principalmente, iOS y Android. Al momento de su nacimiento, React Native salió como un entorno de uso gratuito para desarrollar con JavaScript aplicaciones compiladas para Android, iOS e, incluso en aquel entonces, para el sistema de Windows Mobile, un SO para los teléfonos que comercializaba Microsoft. En 2015 se lanzó como una solución al desarrollo múltiple para la misma aplicación, bajo **licencia MIT**, es decir, licencia de software libre permisiva, con la posibilidad de usarse en aplicaciones privativas.

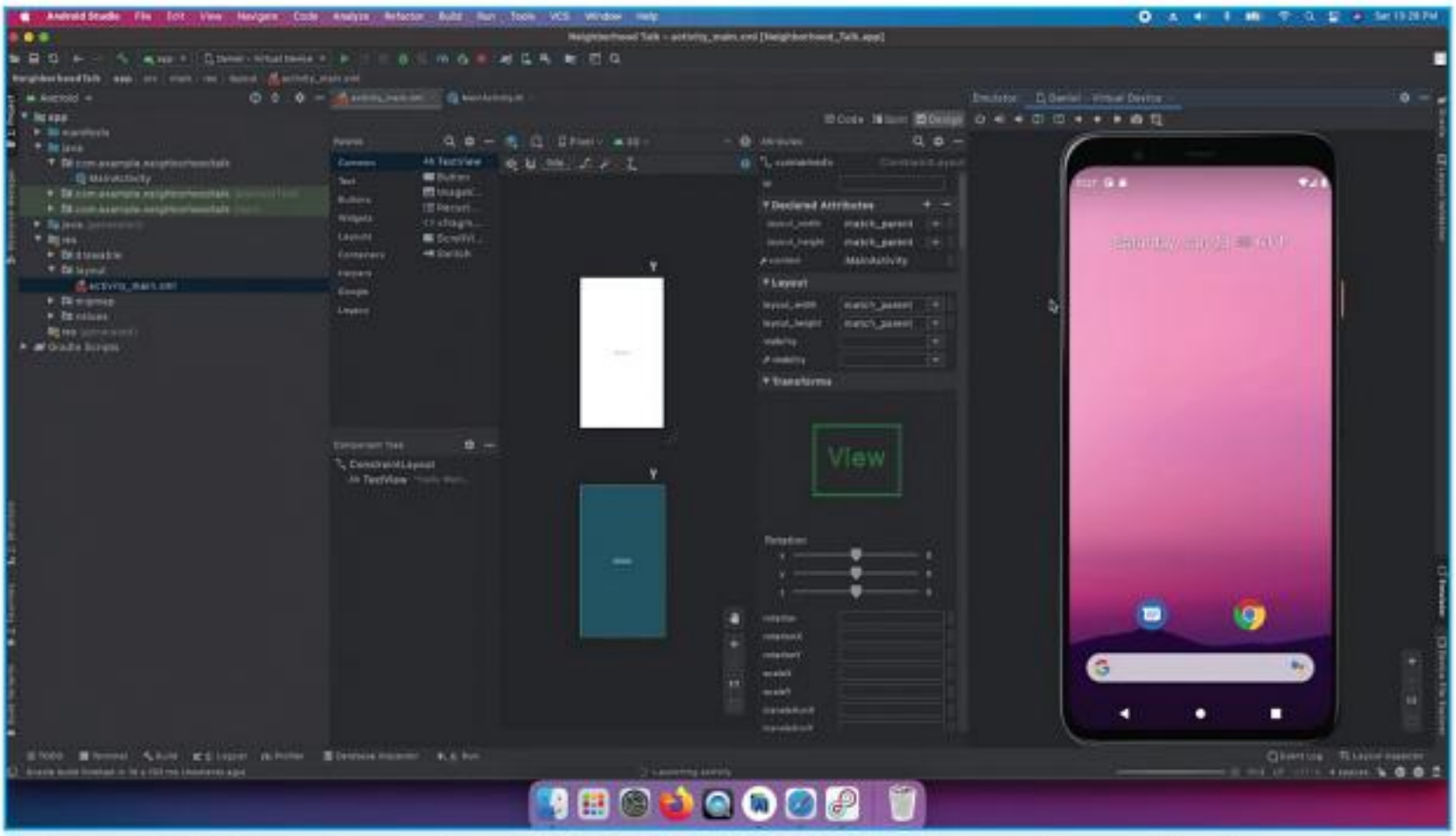


Figura 1.5. *El programa Android Studio es utilizado para el desarrollo de aplicaciones con Java o Kotlin.*

RU

El entorno de trabajo, como su nombre lo indica, está basado en la librería React, ya que conserva su formato de trabajo con archivos de tipo JS y **JSX**. Utiliza el lenguaje JavaScript, retorna marcado **JSX** y emplea componentes en cada parte de la aplicación, que permiten la reutilización y el cambio.
Una aplicación de React para la Web se vería de esta forma:

```
import React from ‘react’;
import ReactDOM from ‘react-dom/client’;
import App from ‘./App’;

const root = ReactDOM.createRoot(document.
getElementById(‘root’));
root.render(
    <App />
);
```

Por su parte, React Native trabaja también con componentes creados por el desarrollador, utilizando la librería de React, pero incluyendo todo el framework con los componentes de React Native.
De esta manera se vería un componente de React Native:

```
import { AppRegistry, Text } from ‘react-native’;
import * as React from ‘react’;

const Title = () => {
    return (
        <Text>Titulo de la app</Text>
     );
}
```

1. React Native

```
export default Title;
```

Como puedes observar, el elemento **<Text>** no existe en ningún caso dentro de la Web, a diferencia de lo que sucede en React, donde escribirías código basado en las etiquetas de **HTML**, o teniendo que crear tus propios componentes antes de utilizarlos. React Native ofrece toda una librería de componentes listos para usar que, luego, compilan dentro de la aplicación de una forma particular. No permite utilizar elementos como **<h1>**, **<h2>**, **<p>**, **<a>** o ** **, sino que tendrás que aprender a usar los componentes creados a partir de la orientación a los dispositivos móviles en cada caso, teniendo elementos de tipo vista, listas con scroll, textos, imágenes, y muchísimo más. Por esta razón, React Native se considera un framework, construido sobre la librería de React. Es por esto que, para aprender a desarrollar aplicaciones de React Native, es requisito previo conocer y manejar al menos en nivel intermedio **JavaScript**, conocer sobre **CSS** y **HTML**, y también tener conocimiento de la librería React.

Entorno de trabajo

Otro factor importante dentro del desarrollo de aplicaciones nativas es el entorno en el cual se trabaja o donde se desarrolla la aplicación. Este factor cobra relevancia dado que es necesario probar el código a medida que se avanza en la programación. Es posible que una aplicación se vea de cierta manera en un dispositivo, pero que no funcione como se espera en otro. Por esta razón, se necesita instalar un entorno de trabajo, el lenguaje a utilizar, compiladores y, también, un emulador que permita en tiempo real visualizar el trabajo que se realiza dentro de la computadora, sin necesidad de compilar la aplicación completa y desplegarla a producción. A diferencia del desarrollo web, que puede simplemente probarse en un navegador,

algo de fácil acceso, una aplicación nativa requiere de un teléfono para instalarse, un archivo APK en caso de Android, el proceso de compilación y la prueba completa. Por este motivo, aun con el uso de Java y Android Studio, existen emuladores que deben instalarse en la computadora para probar el código a medida que se lo crea.

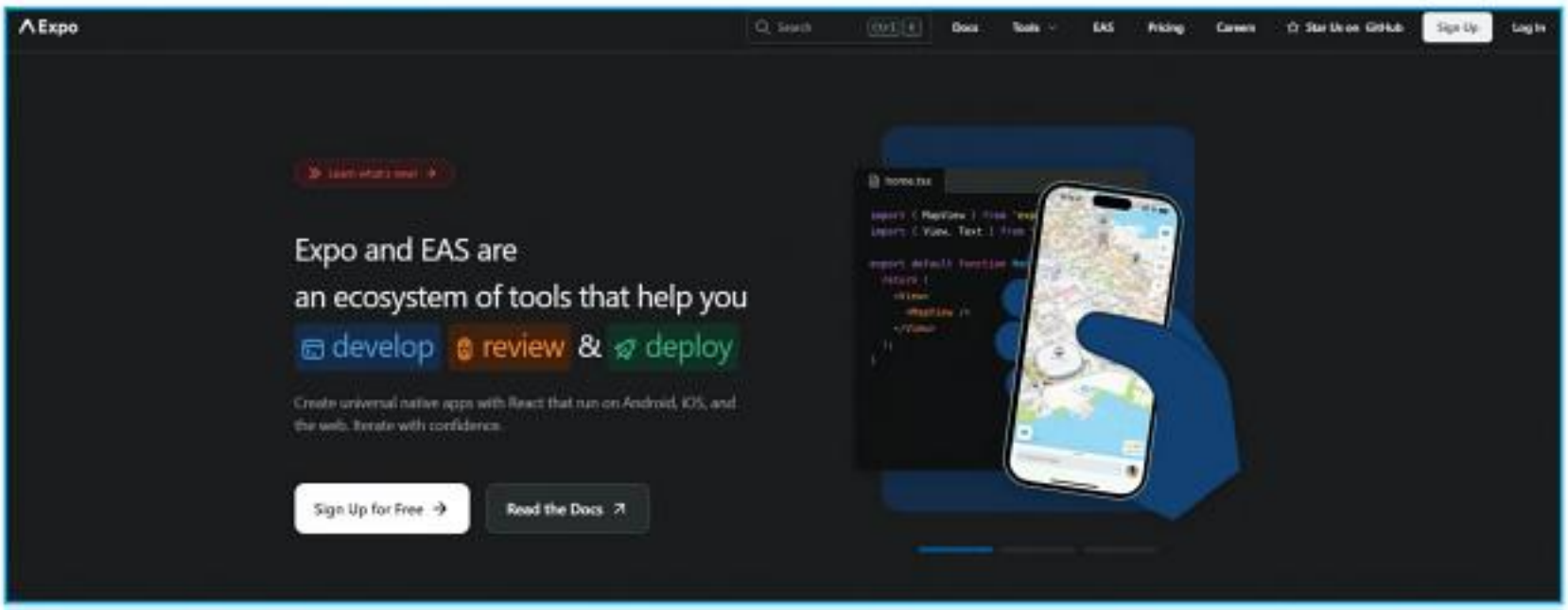


Figura 1.6. *Expo es una plataforma dedicada a facilitar la compilación y el despliegue de apps en la nube.*

Para comenzar a trabajar con React Native, es necesario contar con una computadora que tenga Windows, Linux o MacOS; al menos 8 GB de memoria para ejecutar la compilación de código, el editor y el emulador en paralelo; y que disponga de conexión a Internet. En esta obra mostraremos la instalación de los programas y lenguajes en un entorno Windows.
Como primer paso, dirígete a la página oficial de React Native ingresando a este link, donde verás qué plataformas elegir. Dentro del entorno de React Native hay dos herramientas principales entre las cuales elegir antes de empezar, dado que no es posible cambiar en mitad del proyecto. Se trata de herramientas dedicadas a facilitar el trabajo de emular y, también, las tareas de postdesarrollo, es decir, compilar la aplicación, desplegarla y publicarla.
Por un lado, puedes optar por instalar el **SDK** de Android y React Native mediante **NPM**, el gestor de paquetes y desarrollo para JavaScript y **Node.**

JS, sin ninguna asistencia extra, lo cual significa que dispondrás de la librería completa de React Native junto con todas las librerías externas que desees configurar. Tendrás que usar Android Studio para compilar la aplicación y emularla.



Figura 1.7. *Node y sus entornos de trabajo permiten crear todo tipo de aplicaciones, desde servidores hasta aplicaciones web.*

Por otro lado, está Expo, una plataforma muy popular creada para facilitar el desarrollo de aplicaciones con React Native compilando y emulando con un teléfono móvil a medida que se va programando. En este ejemplo práctico, utilizaremos Expo para facilitar el trabajo, dado que es más simple de practicar con esta herramienta, pero es importante aclarar que no todas las herramientas y librerías que instales en Expo pueden usarse con React Native sin Expo. La plataforma Expo ofrece un inicio rápido y explicación de uso en su página oficial ingresando a este link.

Para empezar, es necesario tener instalado en la computadora local **Node.JS** con **NPM**, que en el caso de la instalación de Windows, pueden descargarse desde este link, donde se instalará tanto el entorno de **NodeJS** como el gestor **NPM** en conjunto. Si estás en un sistema de tipo **Debian**, como **Ubuntu** Linux, puedes instalar Node.JS mediante los

comandos **`sudo apt update`**, luego **`sudo apt install nodejs`** y **`sudo apt install npm`**.

Después de instalar Node.JS, deberías poder acceder a la versión escribiendo en una terminal el comando **`node -v`**.

Una vez instalado **`Node.JS`**, dirígete a la carpeta donde quieras guardar tu proyecto de React Native y allí abre una terminal de CMD o PowerShell, donde deberás ingresar el comando **`npx create-expo-app@latest`** para crear un nuevo proyecto de React Native. Si recibes un error en Windows indicándote la falta de un directorio llamado **`npm`** en tu carpeta Roaming, ve a **`C:\Users\Username\AppData\Roaming\`** y, allí, crea un directorio llamado **`npm`**.

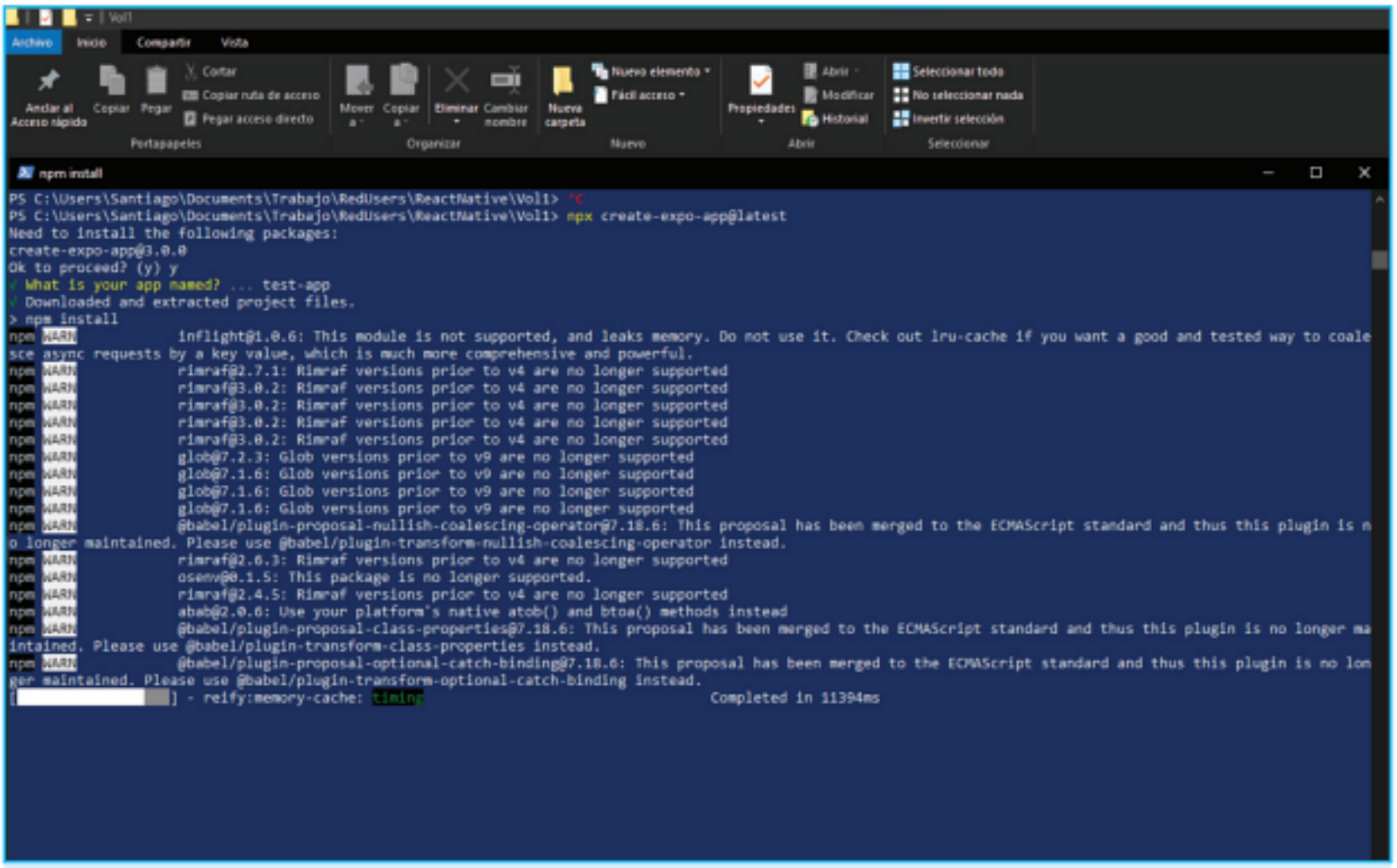


Figura 1.8. *La terminal te indicará que se instalarán algunos paquetes de forma global, y luego te pedirá un nombre para tu aplicación.*

Una vez que indiques qué nombre deseas darle al proyecto, se instalarán todas las dependencias necesarias y podrás comenzar a trabajar.

En la consola verás algunos comandos útiles para usar con Expo y poder ver tu aplicación en tiempo real.

```
added 1569 packages, and audited 1570 packages in 3m

147 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

› Your project is ready!

To run your project, navigate to the directory and run one of the following npm commands.

- cd test-app
- npm run android
- npm run ios # you need to use macOS to build the iOS project - use the Expo app if you need to do iOS development without a Mac
- npm run web
PS C:\Users\Santiago\Documents\Trabajo\RedUsers\ReactNative\Vol1>
```

Figura 1.9. *La terminal te indicará que se instalarán algunos paquetes de forma global, y luego te pedirá un nombre para tu aplicación.*

Para comenzar a escribir código, tendrás que abrir el proyecto en Visual Studio Code o en el editor de texto que quieras usar. VS Code es una herramienta útil ya que permite trabajar con extensiones o plugins que facilitan la tarea, y también es multiplataforma, con lo cual puede utilizarse en cualquier sistema operativo.
Una vez abierto el editor de código, podrás correr en la terminal el comando **`npx expo start`**, que comenzará a correr el compilador y mostrará cómo puedes emular tu proyecto, para ver su plantilla inicial.

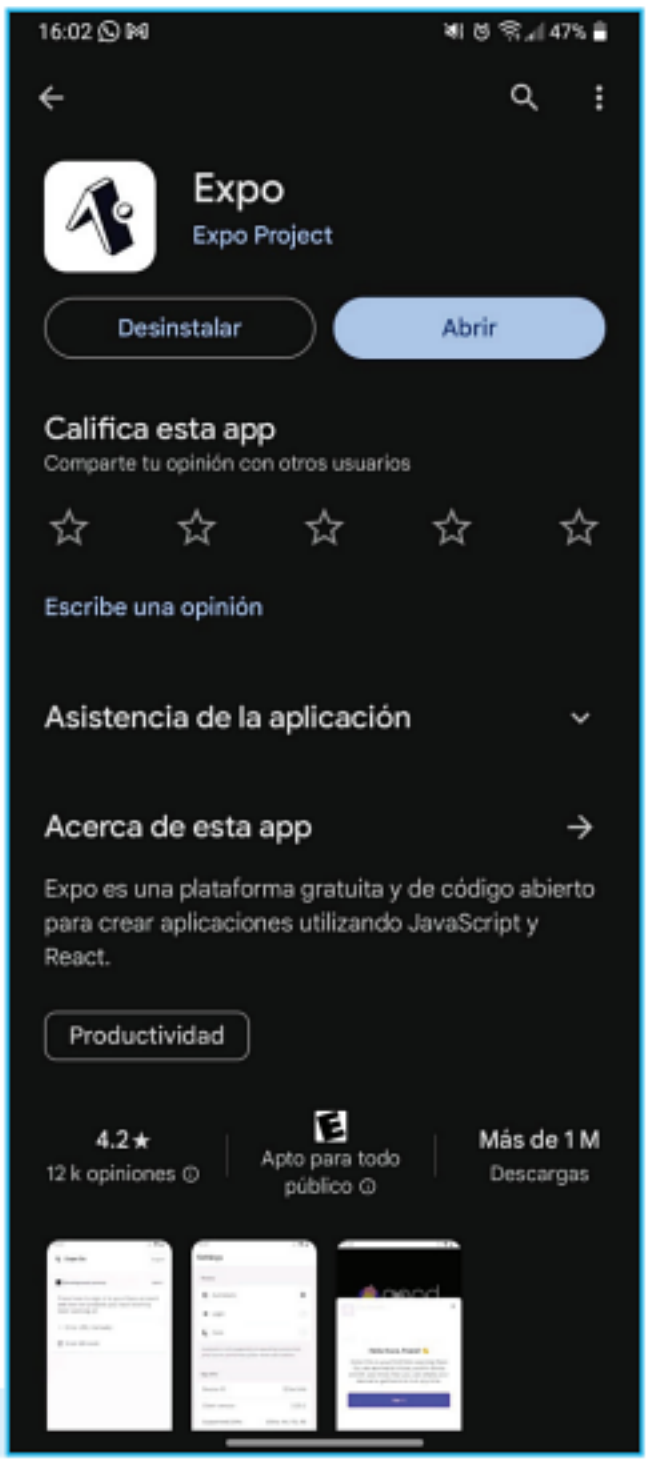


Figura 1.10. *La aplicación Expo Go está en la App Store y en Google Play Store.*

Si todo va bien y las instalaciones previas se ejecutaron correctamente, verás un código QR dentro de la terminal, que te indica que puedes instalar un emulador en tu teléfono portátil. Esto es una ventaja de Expo, ya que tienes dos opciones a la hora de probar y correr el código. Por un lado, puedes instalar un emulador de Android en tu computadora, lo cual consumirá más recursos. Por otro lado, puedes recurrir a una aplicación llamada Expo Go, desarrollada por Expo para emular en tiempo real tu aplicación dentro del teléfono. En este punto, trabajaremos con la aplicación móvil de Expo Go, y en capítulos posteriores, veremos cómo instalar un emulador junto con Android Studio y el SDK de Android para emular la aplicación.

Si tienes un dispositivo Android, dirígete a Google Play Store y descarga la aplicación Expo en tu teléfono. Una vez hecho esto, ingresa en ella y, desde allí, escanea el código QR que se ve en la terminal para abrir la plantilla base de React Native con Expo; se mostrarán algunos textos e imágenes como inicio rápido.

Desde mediados de 2024, la instalación de React Native con Expo incluye la instalación del navegador con **Tabs**, tema que profundizaremos más adelante, pero diremos que, básicamente, permite navegar entre pantallas dentro de la aplicación. Una vez instalada y abierta la aplicación en Expo en tu teléfono, podrás ver algo similar a la imagen de la siguiente figura:

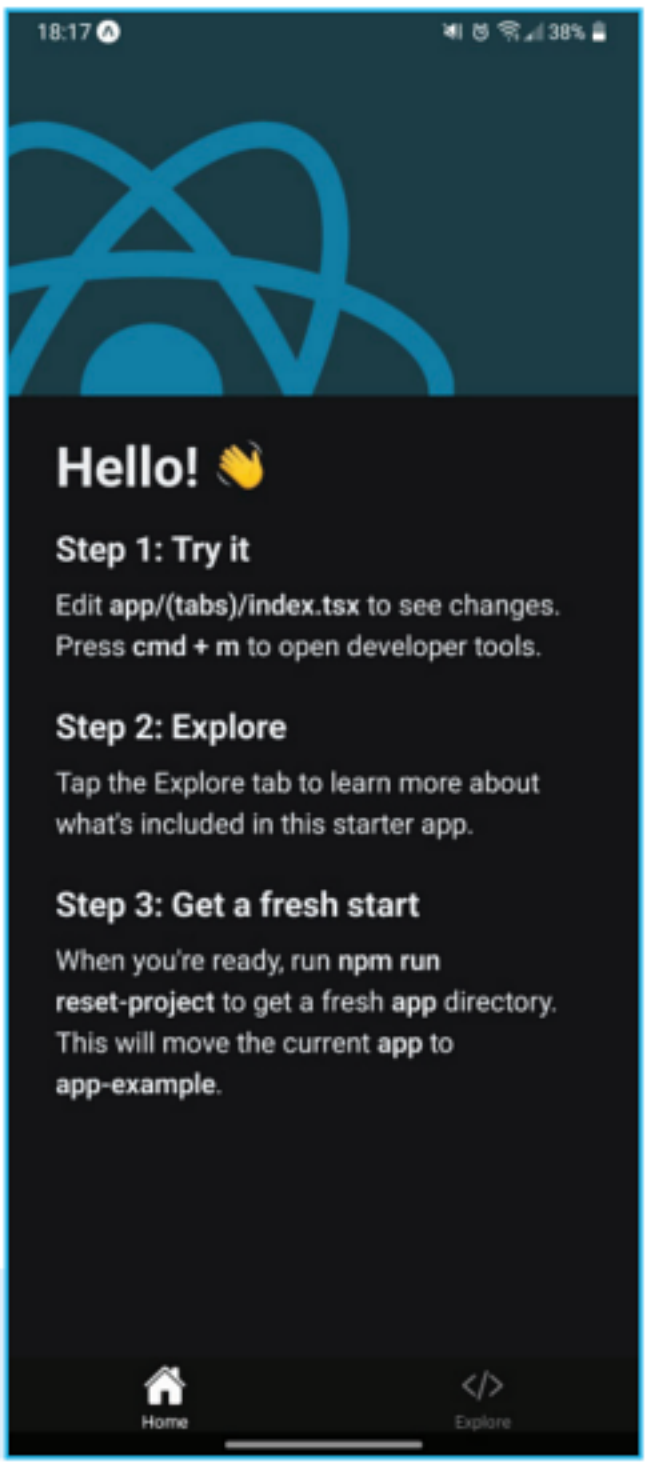


Figura 1.11. *La plantilla básica de React Native con Expo muestra algo de texto e indicaciones sobre cómo comenzar a trabajar.*

Ahora que has instalado la aplicación y configuraste el entorno, es momento de empezar a programarla.

Comenzar a desarrollar

Material adicional en PAG. 04

Ya instalaste React Native y Expo; es momento de comenzar a trabajar en tu primera pantalla. React Native y las aplicaciones móviles funcionan con un sistema de pantallas, y la aplicación, por defecto, abrirá el archivo **`index.tsx`** en la pantalla definida en la carpeta **`app`**, dentro de la carpeta **`tabs`**. En este punto, es importante tener en cuenta que, de manera predefinida, no utilizarás JavaScript plano, sino que, al instalar React Native, se trabaja con un lenguaje subconjunto llamado TypeScript. Se trata de una extensión de JavaScript, creada para trabajar con tipado estricto, lo que te permite especificar el valor de las variables y elementos en cada declaración. En JavaScript nativo, una variable puede definirse y el tipo de dato puede cambiar en el futuro, pero en TypeScript, puedes apoyarte en el tipo de dato e, incluso, definir tus propias estructuras de datos, algo que resulta muy conveniente a la hora de trabajar, por ejemplo, con peticiones externas a servidores, y que te ayudará también a detectar errores en el futuro antes de compilar el código.

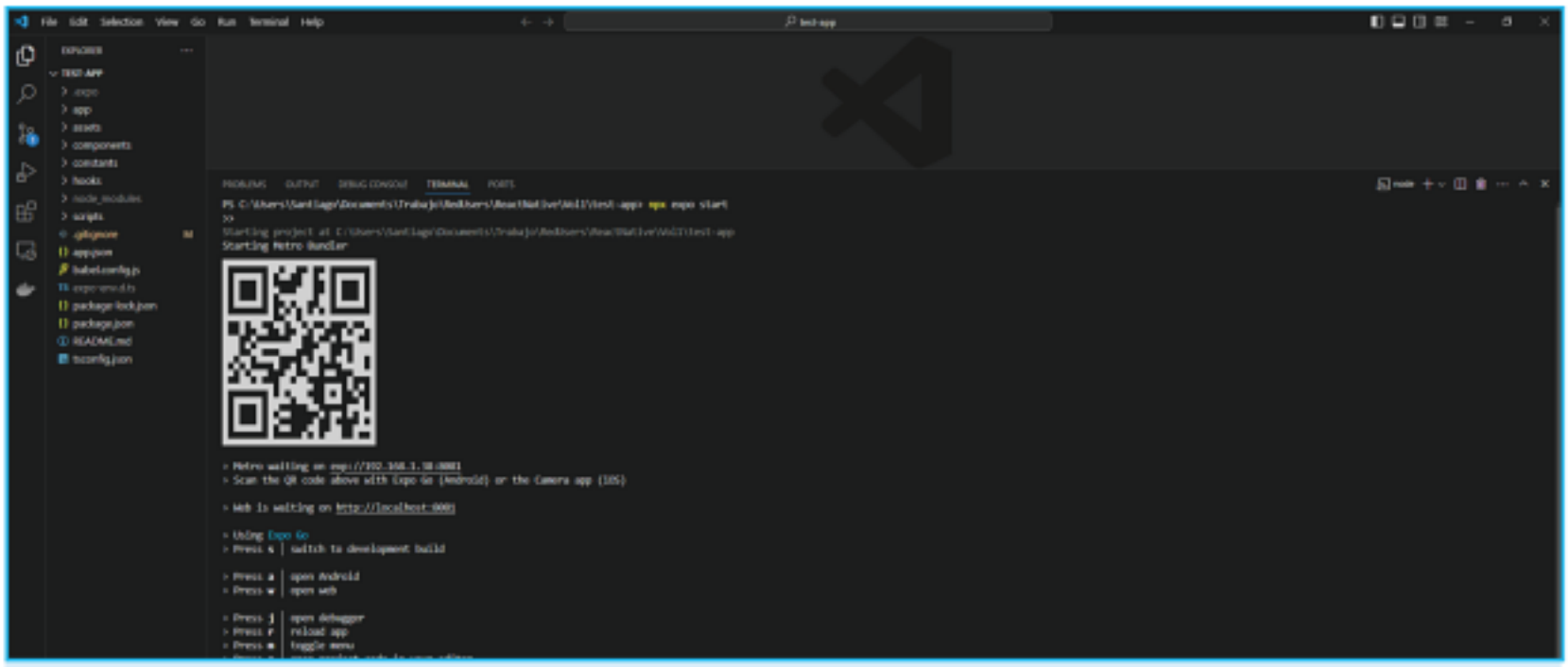


Figura 1.12. *Expo permite abrir el emulador en tu celular o computadora.*

Para comenzar, dirígete al archivo **index.tsx** y edita la línea 19, donde se encuentra el siguiente código:

```
<ThemedText type=”title”>Welcome!</ThemedText>
```

Cambia el texto dentro del componente por Hola Mundo u otro; inmediatamente verás el cambio en el emulador.
Ahora, elimina por completo el contenido de la instrucción **return()** y también quita los **imports** que no se utilicen, ya que comenzarás a trabajar con la plataforma desde cero. El contenido del archivo debería lucir de la siguiente forma:

```
import { Image, StyleSheet, Platform } from ‘react-
native’;

export default function HomeScreen() {
  return (

  );
}

const styles = StyleSheet.create({
  titleContainer: {
    flexDirection: ‘row’,
    alignItems: ‘center’,
    gap: 8,
  },
  stepContainer: {
    gap: 8,
```

```
    marginBottom: 8,
  },
  reactLogo: {
    height: 178,
    width: 290,
    bottom: 0,
    left: 0,
    position: 'absolute',
  },
});
```

Analizando un poco el código de este archivo, puedes notar que se ve muy parecido a React, pero en este caso, no podrás utilizar elementos nativos de HTML.

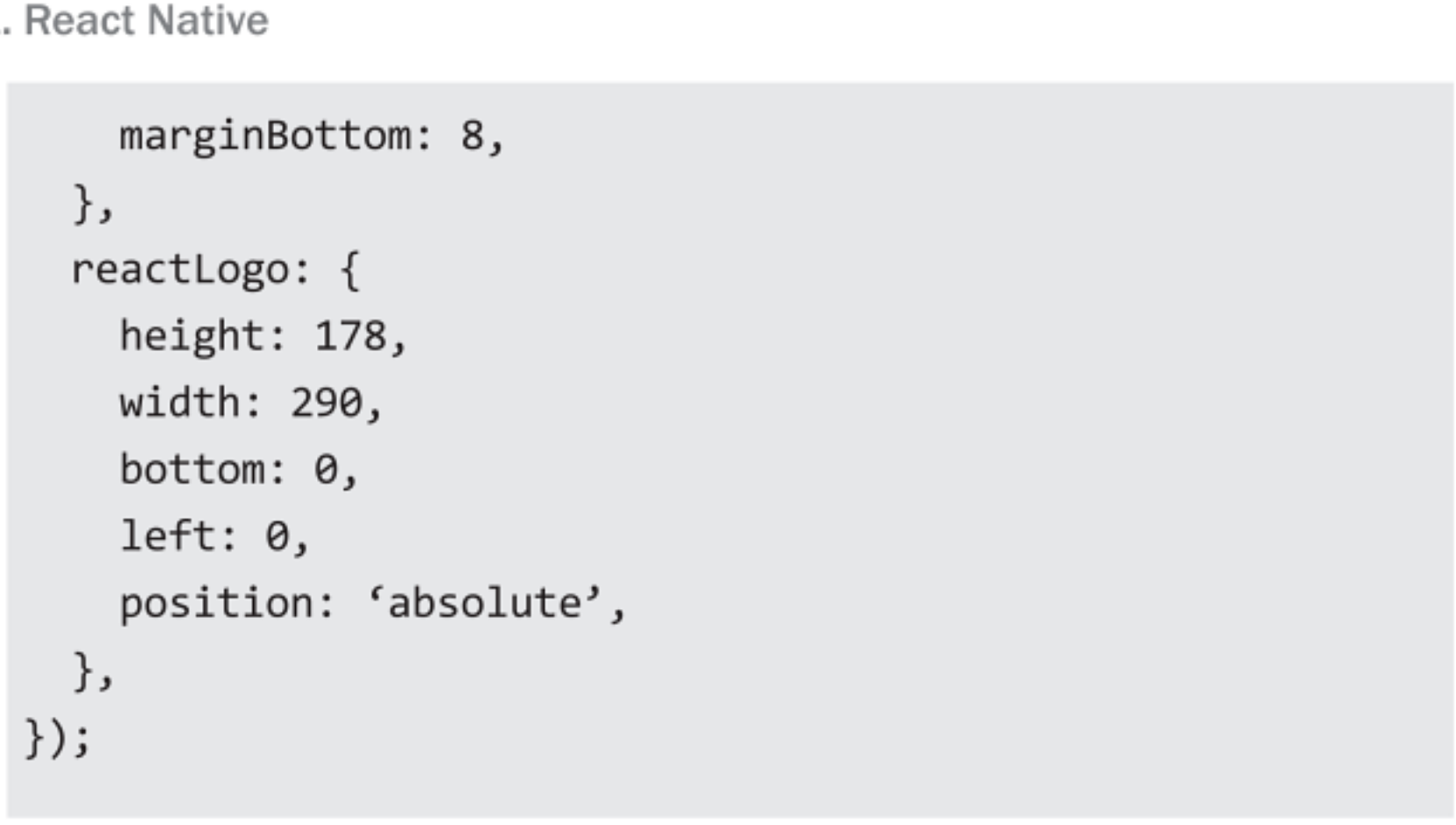


Figura 1.13. *Tu aplicación de React Native debería verse de esta manera.*

En la sección inferior, verás un objeto de JavaScript en una constante llamada **styles**. Como puedes observar, en esta sección se definen los estilos de la pantalla. Dentro de React Native no se utiliza CSS, sino que puedes utilizar un objeto con propiedades similares a las de CSS. Cada objeto dentro del elemento **styles** trabaja como una clase, en la cual definirás propiedades. Es posible usar características como **margin** o **padding**, pero en vez de la notación de guion como en CSS, tendrás que recurrir a la notación de camel case, es decir, utilizar **paddingTop**, **paddingVertical**, **paddingLeft**, **paddingRight**, **marginTop**, **marginBottom** y **marginHorizontal**, entre otras. Las características de los estilos dependen de qué componentes utilices, al igual que en HTML y CSS. El margen o el relleno pueden funcionar de distinta manera si se usan en un elemento o en otro. Para definir estilos, es necesario utilizar la clase **StyleSheet** y llamar al método **create()** para definir la hoja de estilos. Por ahora, elimina todos los elementos dentro del objeto de estilos.

Dentro de la instrucción **return()** deberás colocar los elementos que React Native proporciona para trabajar. Incluye el siguiente código en el archivo **index.tsx**:

```
import { Image, StyleSheet, Platform, View, Text } from
‘react-native’;

export default function HomeScreen() {
  return (
    <View>
      <Text>
        Mi primera pantalla en React Native.
      </Text>
    </View>
  );
```

```
}

const styles = StyleSheet.create({

});
```

Una vez hecho esto, la pantalla pasará a color negro y no verás más componentes. Agrega los siguientes cambios:

```
import { Image, StyleSheet, Platform, View, Text } from
‘react-native’;

export default function HomeScreen() {
  return (
    <View style={styles.container}>
      <Text>
        Mi primera pantalla en React Native.
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: ‘center’,
    alignItems: ‘center’,
    backgroundColor: “white”,
  }
```

```
});
```

Aquí añadirás una clase similar a como lo harías en CSS, un conjunto de propiedades para la clase **`container`**, o más bien, el objeto **`container`**, el cual tendrá la propiedad **`flex`** configurada en 1, y las propiedades **`justifyCenter`** y **`alignItems`**, que funcionan de modo similar a las propiedades de **`Flexbox`** de CSS, con el mismo nombre. Además, se configura el fondo del componente en blanco. Al igual que en CSS, podrás reutilizar este mismo objeto **`container`** para todos los componentes que desees; solo tienes que indicárselo por medio de la propiedad **`style={styles.container}`**.

Ahora, en el emulador, verás que la pantalla está en blanco y que en el medio aparece el texto Mi primera pantalla en React Native. Como puedes notar, se utilizan dos componentes en la pantalla, **`View`** y **`Text`**. El elemento **`Text`** se usa para renderizar en la pantalla texto plano, ya sea títulos, párrafos o cualquier otro que necesites. Por otro lado, puedes pensar en el componente **`View`** como un reemplazo de la etiqueta HTML **`<div>`**. Este elemento permite configurar dentro de otros un contenedor para imágenes, texto, listas, botones, etc. Como puedes imaginar, no es posible utilizar elementos como **`<button>`** o **`<a>`**. En su lugar, tendrás que usar otros, como **`<Button>`** o **`<TouchableOpacity>`**. El componente de React Native **`Button`** es un botón genérico que permite configurar un comportamiento en particular tras el evento **`onPress`**, es decir, cuando el usuario presiona el botón. Por su parte, **`TouchableOpacity`** es un elemento que puedes configurar con más estilos y características que **`Button`**.

Agrega el siguiente botón dentro del código, de la siguiente manera:

```
export default function HomeScreen() {
  return (
```

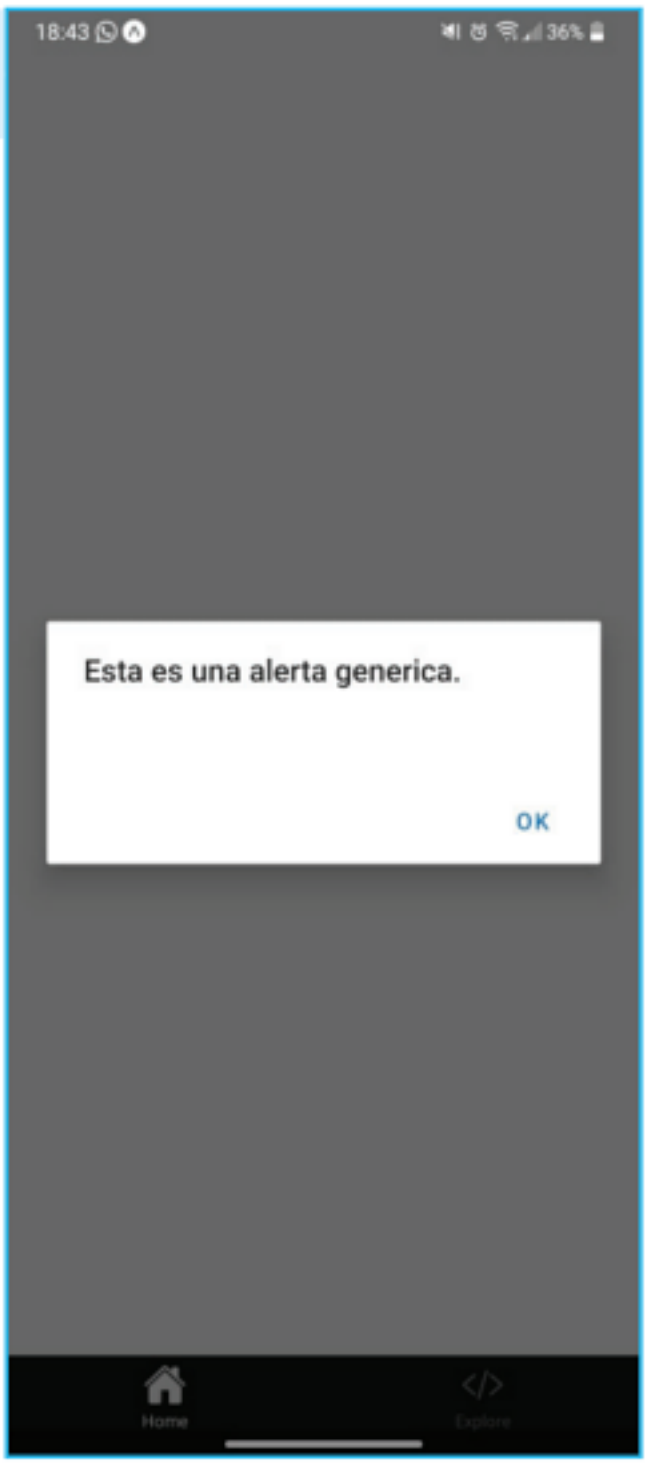
```
    <View style={styles.container}>
      <Text>
        Mi primer pantalla en React Native.
      </Text>
      <Button
        title='Mi boton'
        onPress={() => Alert.alert('Esta es una alerta
generica.')}
      >
      </Button>
    </View>
  );
}
```

Como puedes ver, se agrega un botón con el componente **`Button`**, dentro de la instrucción **`return()`**, y luego se configuran dos **`props`** en el mismo componente para indicar qué texto debería tener el botón, mediante la propiedad **`title`**; con el evento **`onPress`**, se envía una función.
En la sección superior, dentro de los **`imports`**, agrega los siguientes componentes:

```
import { StyleSheet, View, Text, Button, Alert } from 'react-native';
```

De esta forma, al tocar el botón en la pantalla, verás una alerta genérica con el texto que indicaste como parámetro de la función (Figura 1.14.).

Figura 1.14. *Al presionar el botón, verás cómo se lanza una alerta dentro de la pantalla.*



Como puedes ver, si no defines ambas props, el componente falla y lanzará un error de compilación. Al igual que con React, existen propiedades que son requeridas y otras que pueden o no definirse, es decir, que son opcionales.

Además de declarar el título o el evento que lanza el botón, puedes declarar otras características, como el color, si está deshabilitado o no, o incluso, una propiedad llamada **`accessibilityLabel`**, que toma como parámetro un **`String`** y permite definir un texto que será leído por los lectores de pantalla en el caso de personas con visibilidad reducida.

Añade al botón las siguientes propiedades:

```
<Button
     title=’Mi boton’
     onPress={() => Alert.alert(‘Esta es una alerta
               generica.’)}
     accessibilityLabel=”Mi boton”
     disabled={false}
     color={‘red’}
>
</Button>
```

Una vez hecho esto, agrega al título la siguiente propiedad:

```
<Text style={styles.title}>
    Mi primer pantalla en React Native.
</Text>
```

Y luego, agrega al objeto de estilos el siguiente código:

```
title: {
    marginVertical: 15,
},
```

Puedes definir el margen vertical de los elementos de forma más rápida con esta propiedad, evitando tener que definir el margen superior e inferior en dos líneas, como lo harías en CSS.

En los siguientes capítulos, aprenderás a trabajar con estilos más avanzados, pantallas dentro de la aplicación, y otros componentes como imágenes, listas y elementos con **`scroll`**.

Actividades

A continuación se presentan las preguntas y los ejercicios que deberías saber responder y resolver para considerar aprendido el capítulo.

Test de autoevaluación

1. *¿Qué sistemas operativos móviles existen en la actualidad?*
2. *¿En qué núcleo está basado Android? ¿Qué tipo de licencia posee Android?*
3. *¿Qué ventajas trae el uso de React Native?*
4. *¿Qué es Expo?*
5. *¿Para qué es necesario el uso de un emulador?*

Ejercicios prácticos

1. *Ubica dentro de tu proyecto la pantalla Explore.*
2. *Edita la pantalla explorar y cambia su contenido indicando el cambio de pantalla.*
3. *Intenta editar las tabs inferiores con distintos textos en cada caso.*
4. *Agrega fondos distintos para cada pantalla.*

Capítulo 02



Estilos

En el capítulo anterior, trabajamos en la instalación y configuración de React Native, para comenzar a desarrollar una aplicación destinada a dispositivos móviles utilizando uno de los lenguajes de programación más demandados en los últimos tiempos, JavaScript.

Estilos y contenedores / 35

Navegación y pantallas / 44

Actividades / 58

- Test de autoevaluación / 58
- Ejercicios prácticos / 58

Estilos y contenedores

Material adicional en PAG. 04

Como vimos en páginas previas, en el desarrollo de una aplicación móvil en React Native, no es posible utilizar CSS para definir el estilo de los componentes de cada pantalla. En este capítulo, aprenderás a utilizar los estilos provistos por la librería para dar un diseño único a tu aplicación.
Si bien React Native se basa en React, cuenta con su propio sistema de componentes y de diseño, que se parece muchísimo a lo que ya usaste en el pasado con CSS. Las propiedades en React Native que definen el diseño se declaran en un objeto que se crea con la clase **`StyleSheet`** de React Native, y llevan nombres muy similares a los de CSS, sumándose algunas declaraciones muy útiles para acortar el código.
Además, el comportamiento de los elementos se define mediante un sistema basado en las propiedades **`Flexbox`** de CSS, lo que permite configurar con facilidad cómo se ven los distintos componentes en la pantalla. Para comenzar a usar **`Flexbox`** y comprender de qué manera se aplican estas propiedades, dirígete al archivo **`explore.tsx`** de tu aplicación, dentro de la carpeta **`(tabs)`**, y allí edita el contenido del componente para que luzca de este modo:

```
import {
     StyleSheet,
     Text,
     View } from ‘react-native’;

export default function TabTwoScreen() {
     return (
     <View>
          <View>
               <Text>Row 1</Text>
          </View>
```

```
            <View>
                  <Text>Row 2</Text>
            </View>
            <View>
                  <Text>Row 3</Text>
            </View>
      </View>
      );
}
```

En este nuevo componente, como puedes ver, definirás un componente **<View>** dentro de la pantalla para que englobe todo el contenido. En su interior, se definen otros tres elementos también de tipo **View**.

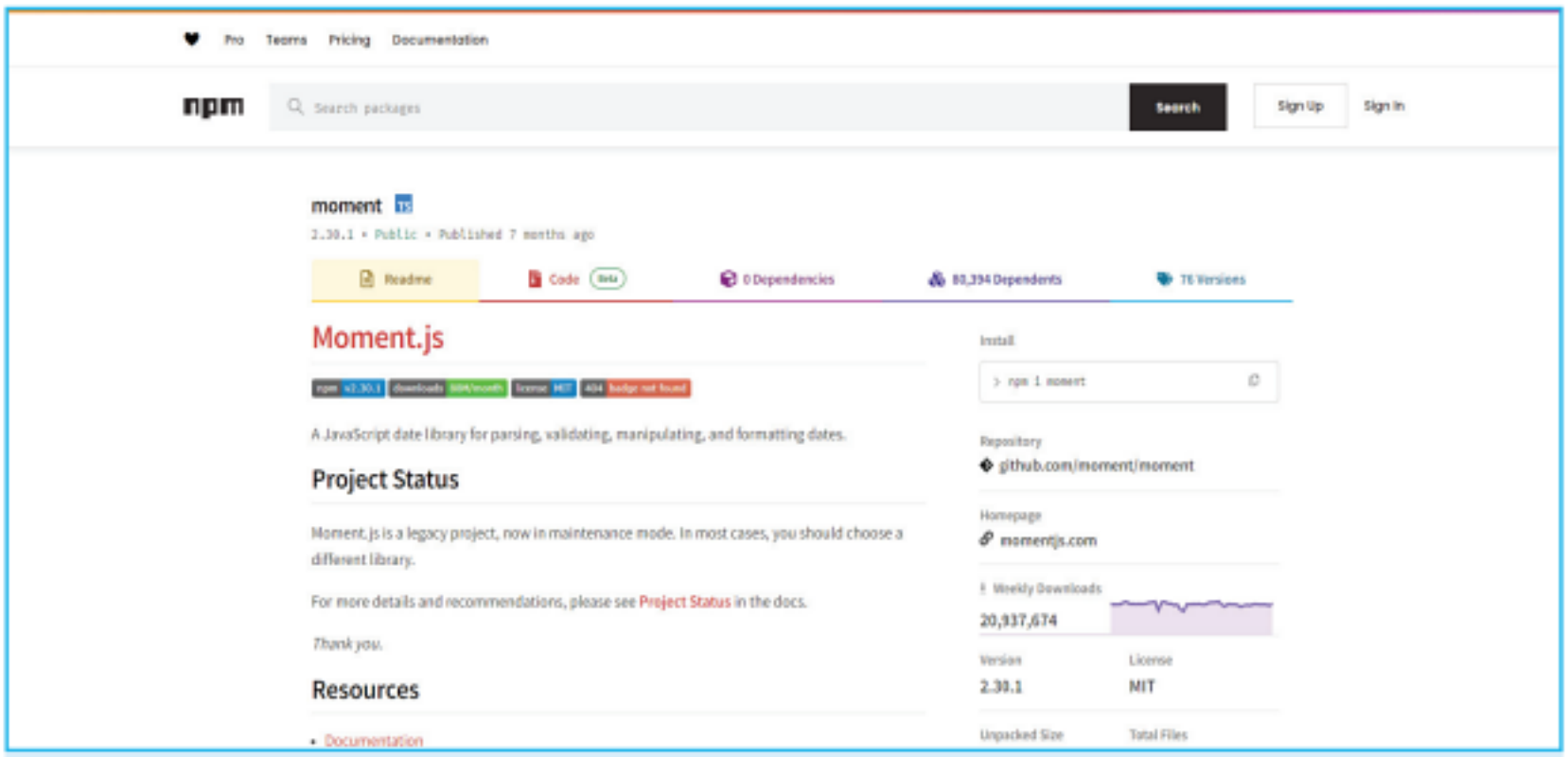


Figura 2.1. *Las librerías de NPM son muy útiles para realizar operaciones comunes, como formatear fechas, por ejemplo, con Moment.JS.*

En este caso, la intención es mostrar tres filas de elementos en la pantalla, todas del mismo tamaño y con un fondo que las distinga. Agrega las siguientes propiedades de estilos debajo en el objeto **styles**:

```
const styles = StyleSheet.create({
     container: {
          flex: 1,
          backgroundColor: ‘white’,
          marginVertical: 25
     },
     row: {
          flex: 1
     },
     dobleRow: {
          flex: 2
     },
     redBackground: {
          backgroundColor: ‘red’,
     }
});
```

En este objeto se define una clave llamada **`container`** y, en su interior, tres propiedades, **`flex`**, **`marginVertical`** y **`backgroundColor`**. La última ayuda a definir el color de fondo de un componente, en tanto que la segunda se utiliza para establecer el margen vertical y permite definir en una única propiedad **`marginBottom`** y **`marginTop`**. Esta propiedad toma como valor un número, de la misma forma que todas las variantes de margen y también las de relleno o **`padding`**. Por último, se especifica la propiedad **`flex`**. Flexbox funciona de manera muy parecida en React Native y dentro de la Web, con ciertas excepciones, dado el **`layout`** de los elementos dentro de un sistema de aplicación móvil. En el caso de React Native, **`flex`** especifica que un elemento padre obliga a sus elementos hijos a colocarse como **`column`** en vez de como **`row`**, como lo haría dentro de la Web; por otro lado, **`alignContent`** se define como **`flex-start`** en vez de **`strech`**.

2. Estilos

Flex define de qué modo los elementos deben llenar la pantalla o su contenedor, dependiendo de qué eje se haya definido. En este caso, se define un contenedor con **flex: 1**, y luego, una clase u objeto **row**, también con **flex: 1**. Si le das al contenedor estas reglas como estilos, deberías ver en la pantalla de tu teléfono o emulador tres filas de contenido, una bajo la otra, con el mismo tamaño.

```
export default function TabTwoScreen() {
    return (
    <View style={styles.container}>
        <View style={styles.row}>
            <Text>Row 1</Text>
        </View>
        <View style={[
            styles.row, styles.redBackground]}>
            <Text>Row 2</Text>
        </View>
        <View style={styles.row}>
            <Text>Row 3</Text>
        </View>
    </View>
    );
}

const styles = StyleSheet.create({
    container: {
        flex: 1,
        backgroundColor: 'white',
        marginVertical: 25
    },
```

```
    row: {
        flex: 1
    },
    dobleRow: {
        flex: 2
    },
    redBackground: {
        backgroundColor: 'red',
    }
});
```

Como puedes ver, también es posible definir en un mismo elemento más de una clase u objeto de estilos. El segundo elemento **View** tiene como regla de estilos **redBackground**, que en su interior ha definido el fondo en color rojo. De esta forma, defines un arreglo de objetos de estilos para que tome todas las características.

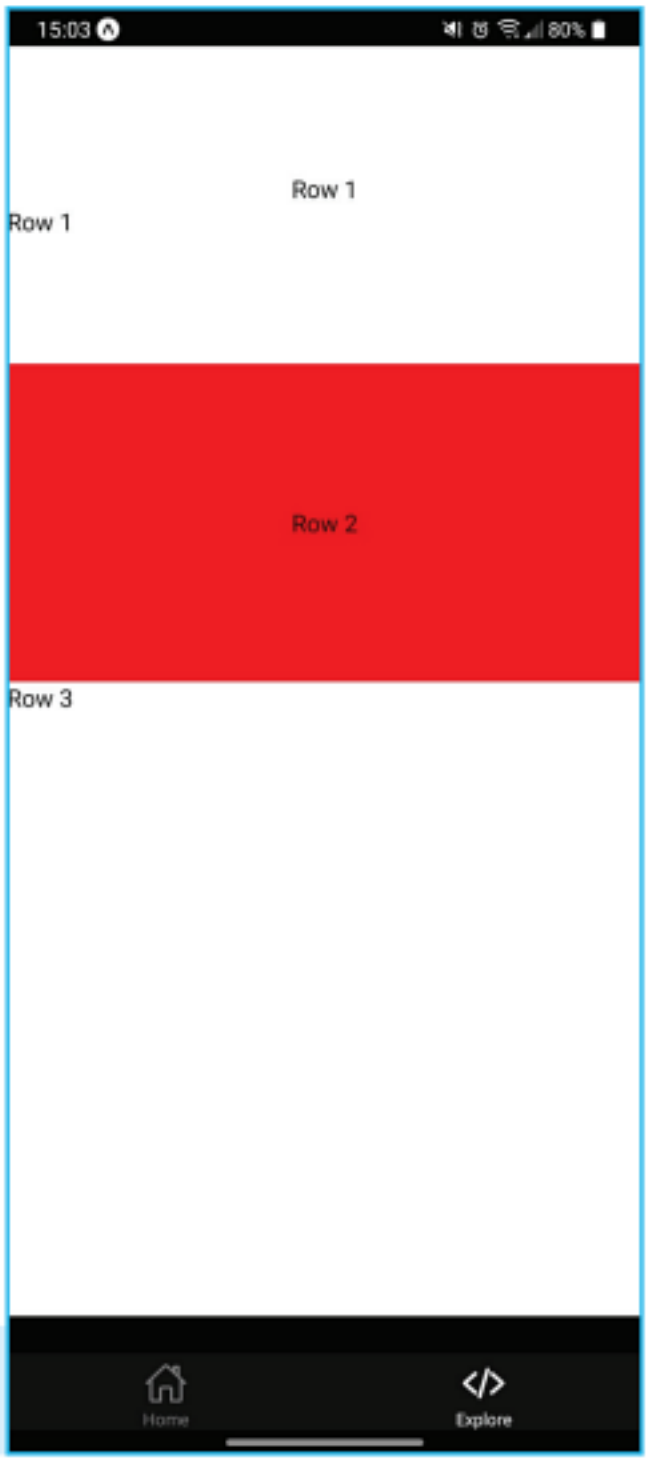


Figura 2.2. *Flex está basado en las propiedades de Flexbox de CSS.*

2. Estilos

Ahora, intenta cambiar el estilo de la tercera fila de elementos, y dale como estilos la clase **dobleRow**. Esto debería indicar que, a diferencia de sus componentes hermanos, esta fila tendrá el doble de tamaño:

```
<View style={styles.container}>
    <View style={styles.row}>
        <Text>Row 1</Text>
    </View>
    <View style={[
        styles.row, styles.redBackground]}>
        <Text>Row 2</Text>
    </View>
    <View style={styles.dobleRow}>
        <Text>Row 3</Text>
    </View>
</View>
```

En tu pantalla, deberías ver que la última fila tiene el doble de tamaño. Esto se debe a que el sistema de **flex** define el espacio dependiendo de la suma de los elementos en el mismo contenedor. En este caso, tienes tres elementos, dos de ellos tienen un punto y el tercero dos, con lo cual la suma resulta en cuatro. El tercer elemento ocupa la mita del espacio, ya que dos es la mitad del espacio.
También es posible cambiar la orientación de los elementos dentro de un contenedor en la pantalla, para lo cual puedes variar entre:

```
container: {
    flex: 1,
    flexDirection: “column”,
```

```
    backgroundColor: ‘white’,
    marginVertical: 25
},
```

O utilizar:

```
container: {
    flex: 1,
    flexDirection: “row”,
    backgroundColor: ‘white’,
    marginVertical: 25
},
```

Incluso, es posible usar las variantes de Flexbox para ordenar los elementos de forma contraria a como están definidos:

```
container: {
    flex: 1,
    flexDirection: “column-reverse”,
…
}
```

O también:

```
container: {
    flex: 1,
    flexDirection: “row-reverse”,
```

```
...
}
```

De esta manera, los elementos se ordenarán de forma inversa a como son declarados.
También es posible definir el alineamiento de los elementos dentro de la grilla por medio de las propiedades de **`Flexbox`**, utilizando **`justify-center`** y **`alignItems`**. Si lo deseas, puedes definir el alineamiento vertical agregando la propiedad **`justifyContent`**, de la siguiente forma:

```
row: {
    flex: 1,
    justifyContent: “center”,
},
```

Y si quieres agregar alineamiento horizontal, deberás añadir:

```
row: {
    flex: 1,
    justifyContent: “center”,
    alignItems: “center”
},
```

Por supuesto, puedes cambiar los valores de estas propiedades y definir, si lo deseas, el alineamiento al comienzo o al final del contenedor, con las propiedades **`flex-end`** y **`flex-start`**.
Además, puedes agregar un alineamiento directo a un elemento con respecto a su contenedor padre, con la propiedad **`alignSelf`**. Para

hacerlo, incluye una línea más de texto a la primera fila de elementos, de esta manera:

```
<View style={styles.row}>
     <Text>Row 1</Text>
     <Text style={styles.startText}>
          Text start
     </Text>
</View>
```

Luego, agrega a los estilos la siguiente configuración:

```
const styles = StyleSheet.create({
     …
     startText: {
          alignSelf: ‘flex-start’
     },
});
```

Así, el segundo elemento de texto en la primera fila se configura al comienzo de su contenedor.
Como puedes observar, React Native aplica a su manera muchas de las características de la programación qué usarías en React para una aplicación web, o incluso, trabajando con CSS y HTML básico, pero orientado a un dispositivo móvil y con la posibilidad de compilarlo a una aplicación desplegable para un marketplace de aplicaciones, una opción muy interesante y demandada dentro del desarrollo de software.

Navegación y pantallas

Material adicional en PAG. 04

A la hora de trabajar en una aplicación móvil, es importante entender que el usuario debe ser capaz de navegar entre las distintas pantallas que la componen para poder distribuir el contenido. Si bien una página puede tener mucho contenido y permitirle al usuario navegar hacia abajo, lo mejor es distribuir la información en distintas pantallas de modo que el visitante obtenga una buena experiencia al usarla.

Dentro del desarrollo web y de aplicaciones, para el programador es vital contar con un prototipo o un diseño hecho previamente para entender la visión final. Tanto si trabajas en un proyecto personal, como si lo haces en una aplicación solicitada por un cliente, la forma óptima de maquetar o desarrollar siempre transita de la mano de un buen planeamiento del proyecto, análisis del producto y un prototipo realizado por un diseñador, un experto en User Experience o que tenga conocimientos similares. Las reglas de UI y UX son fundamentales para el desarrollo en dispositivos usados globalmente, y con diversos tamaños, formatos y capacidades, y pantallas que varían en tamaño de forma muy amplia.

Una de las capacidades más importantes de las aplicaciones es permitir la navegación entre pantallas con botones o por gestos del usuario.

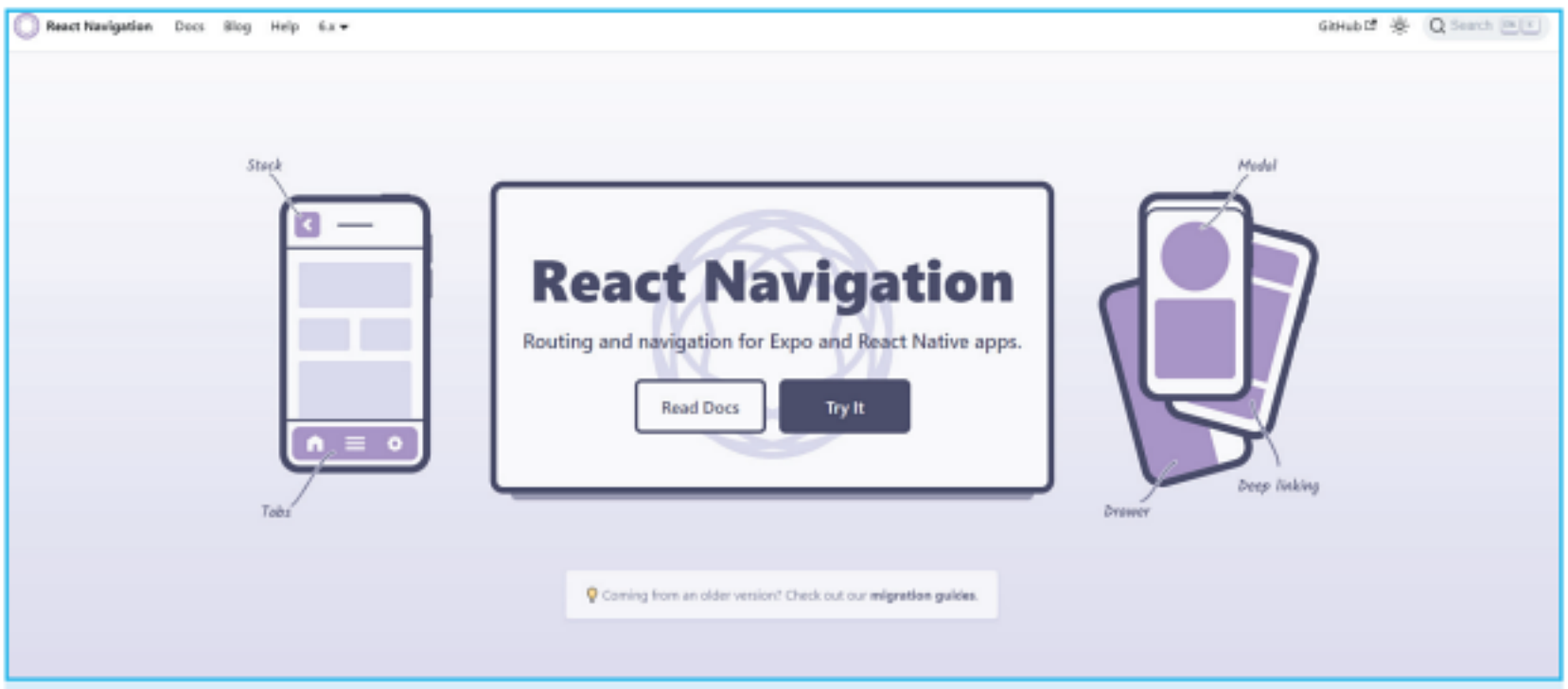


Figura 2.3. *React Navigation se puede instalar por NPM y posee una gran cantidad de funciones para personalizar una aplicación.*

Al instalar Expo con React Native en su última versión, verás en la aplicación generada un sistema con dos pantallas: una home o inicio y una de explorar. Esto es muy útil dado que la mayoría de las aplicaciones, aun las más sencillas, necesitan distintas secciones. Piensa en las pantallas de una aplicación como en los distintos archivos de un proyecto en HTML básico, donde, por medio de elementos HTML `<a>`, puedes navegar por las distintas secciones de tu web. Si has trabajado con `React Router` para la Web, este concepto te resultará fácil de comprender, dado que esta librería permite definir una serie de componentes como secciones de la Web, simulando la navegación entre archivos, pero reemplazando estos por componentes.

En una aplicación móvil, estos archivos se trabajan como en React Router, definiendo un `Layout` general y las pantallas, y compartiendo su estructura. Dirígete al archivo `_layout.tsx` del proyecto dentro de la carpeta `(tabs)`. Allí encontrarás un componente como el siguiente:

```
import { Tabs } from 'expo-router';
import React from 'react';

import { TabBarIcon } from '@/components/navigation/
TabBarIcon';
import { Colors } from '@/constants/Colors';
import {
    useColorScheme }
    from '@/hooks/useColorScheme';

export default function TabLayout() {
    const colorScheme = useColorScheme();

    return (
```

2. Estilos

```
<Tabs
     screenOptions={{
          tabBarActiveTintColor:
          Colors[colorScheme ?? ‘light’].tint,
          headerShown: false,
}}>
     <Tabs.Screen
          name=”index”
          options={{
          title: ‘Home’,
          tabBarIcon: ({ color, focused }) =>
          (
               <TabBarIcon
                    name={focused ? ‘home’
                         : ‘home-outline’}
                    color={color} />
     ),
     }}
/>
<Tabs.Screen
     name=”explore”
     options={{
     title: ‘Explore’,
     tabBarIcon: ({ color, focused }) => (
          <TabBarIcon
               name={focused
               ? ‘code-slash’ :
               ‘code-slash-outline’}
               color={color} />
     ),
```

```
                }}
            />
        </Tabs>
    );
}
```

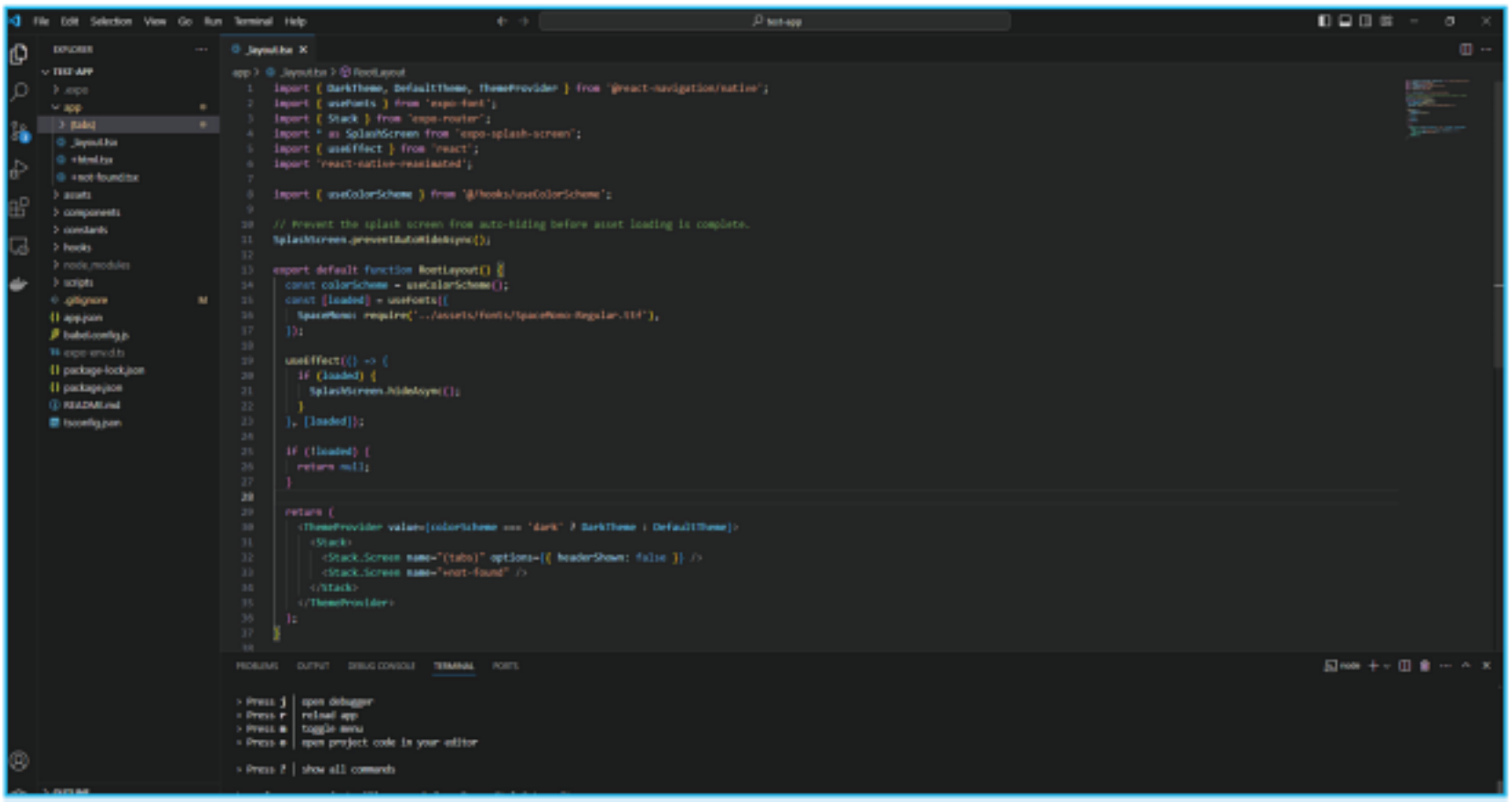


Figura 2.4. *VS Code permite navegar entre archivos con CTRL+clic o buscándolos por nombre desde F1 o con el icono del buscador.*

Como puedes ver, en esta sección se definen algunos componentes de tipo **Tab**. En primer lugar, se utiliza el componente **<Tab>** y luego, en su interior, se definen dos elementos **<Tab.Screen>**, que se importan desde **import { Tabs } from 'expo-router';**. Esta es la manera en la que se declaran y utilizan rutas en React Native. Deberás crear un componente, para luego utilizarlo dentro de un elemento **Tab**, y lograr que sea accesible desde toda la aplicación.

Crea un nuevo elemento **Tab** dentro de la lista de pantallas de tu aplicación, de la siguiente manera:

2. Estilos

```
<Tabs
    screenOptions={{
        tabBarActiveTintColor:
        Colors[colorScheme ?? 'light'].tint,
        headerShown: false,
    }}>
    …
    <Tabs.Screen
        name="list"
        options={{
            title: 'List',
            tabBarIcon: ({ color, focused }) =>
            (TabBarIcon name={focused ?
            'code-slash' : 'code-slash-outline'}
            color={color} />
            ),
        }}
    />
</Tabs>
```

Ahora, tendrás que crear este componente para poder verlo en la pantalla. Crea un archivo llamado **`list.tsx`** dentro de la carpeta **`(tabs)`**, con el siguiente contenido:

```
import { StyleSheet, View, Text, Button, Alert }
    from 'react-native';

export default function AboutScreen() {
    return (
```

```
            <View style={styles.container}>
                  <Text style={styles.title}>
                        List of stores.
                  </Text>
            </View>
      );
}

const styles = StyleSheet.create({
      title: {
            marginVertical: 15,
      },
      container: {
            flex: 1,
            justifyContent: ‘center’,
            alignItems: ‘center’,
            backgroundColor: “white”,
      }
});
```

Ya tienes un nuevo componente listo para usar como pantalla de la aplicación. Dado que llamaste **`List`** a esta nueva sección, será mejor cambiar el icono de la navegación por algo acorde. Como habrás notado, en la parte inferior cada sección utiliza un icono propio para indicar de qué se trata esa sección. En esta pantalla, podrás crear una lista de información genérica, como una lista de sucursales. Si estás utilizando Visual Studio Code, puedes acceder al componente que da un icono a las pantallas de la aplicación, haciendo clic y presionando al mismo tiempo **`CTRL`** en tu teclado o **`command`** en MacOS. Esto te llevará al archivo **`TabBarIcon.tsx`**, que se encuentra en la carpeta **`components`**. Como verás allí, se importa

el componente **Ionicons** de la librería **@expo/vector-icons/Ionicons**, para utilizarse como **isotipos** de cada pantalla. **Ionicons** es una librería Open Source de gráficos, con iconos listos para usar en proyectos de JavaScript, instalable por **npm**. Puedes acceder a la lista completa de iconos en su página oficial, https://ionic.io/ionicons, donde se ofrece una lista con buscador para tus proyectos de código abierto.

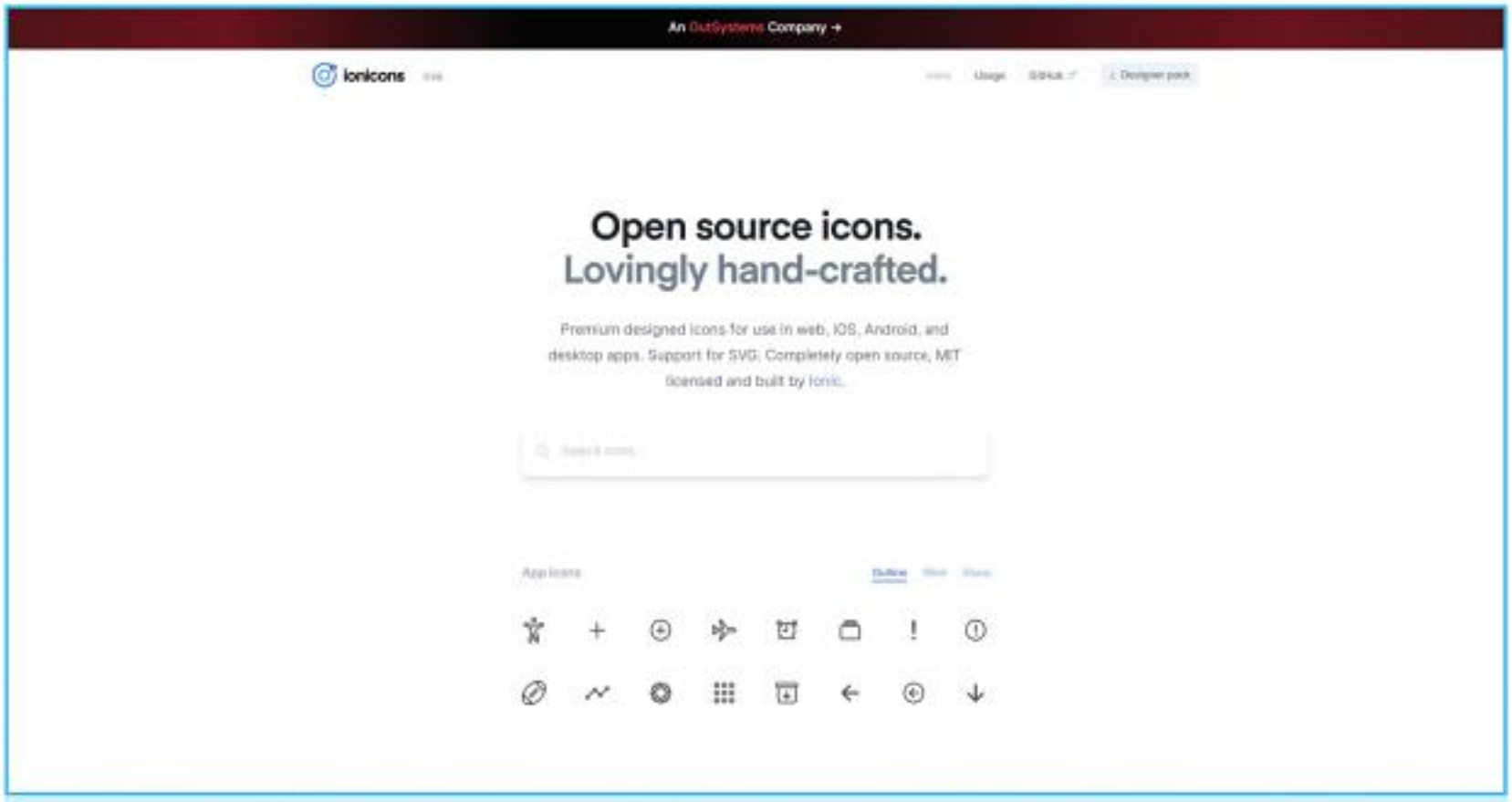


Figura 2.5. *Ionicons es de código abierto y su uso es muy simple.*

Vuelve al archivo **_layout.tsx** de la aplicación y modifica la sección **Tab** que acabas de crear, de esta forma:

```
<Tabs.Screen
    name="list"
    options={{
        title: 'List',
        tabBarIcon: ({ color, focused }) => (
        focused
            ?
```

```
                    <Ionicons size={28}
                         style={[{ marginBottom: -3 }]}
                         name='storefront'
                         color={color}
                    />
               :
                    <Ionicons size={28}
                         style={[{ marginBottom: -3 }]}
                         name='storefront-outline'
                         color={color}
                    />
          ),
     }}
/>
```

Luego, en la parte superior importa la siguiente instrucción:

```
import { Ionicons } from ‹@expo/vector-icons›;
```

Lo que acabas de hacer es importar el componente **`Ionicons`** para aprender cómo implementarlo.
Para utilizar un icono, debes llamar al componente y luego darle algunas características por medio de sus **`props`**.
La primera propiedad que vas a configurar es el tamaño, por medio de **`size`**; luego, puedes definir un estilo según tus necesidades.
Por último, tendrás que seleccionar un color y, lo más importante, indicar qué icono quieres utilizar, por lo que tendrás que configurar **`name`** con el nombre del que desees.
En este caso, lo que se ha declarado es un ternario dependiendo del

valor de la variable **`focused`**. Como puedes intuir, esta variable permite determinar si la pantalla en la que te encuentras es la que pertenece a esta **`Tab`**, y por ende, cambiar su color para indicarle al usuario que ha ingresado o no a la vista **`List`**.



Figura 2.6. *La aplicación ya muestra la nueva pantalla.*

Para que toda la aplicación mantenga una misma lógica, es mejor reutilizar el componente **`TabBarIcon`**. Cambia el código de **`Tab`** de la siguiente manera:

```
<Tabs.Screen
    name=”list”
    options={{
        title: ‘List’,
        tabBarIcon: ({ color, focused }) => (
            <TabBarIcon
                name={focused
```

```
                        ? ‘storefront’
                        : ‘storefront-outline’}
                    color={color} />
                ),
            }}
/>
```

Ahora, podrás navegar a la nueva pantalla y, en el momento en que accedas a ella, verás que el icono cambia de color a blanco completo.



Figura 2.7. *React es una herramienta muy sencilla de utilizar y muy recomendada antes de comenzar con React Native.*

Si bien esto es muy útil, dado que ya puedes navegar entre pantallas, también es necesario poder moverte entre pantallas utilizando botones personalizados. Dirígete al archivo **`index.tsx`** y allí cambia el contenido del componente de la siguiente forma:

```
import { useRouter } from ‹expo-router›;
```

2. Estilos

```
import { StyleSheet, View, Text, Button, Alert }
 from ‘react-native’;

export default function HomeScreen() {
    const router = useRouter();
        return (
            …
        )
}
```

En primer lugar, se importa el **hook useRouter** y, luego, se guarda el elemento **router** para moverse entre pantallas. Este hook está disponible en las versiones de **React Navigation v6** en adelante. Luego, agrega a tu botón el siguiente cambio:

```
<Button
    title=’Mi boton’
    onPress={() => router.push(‘list’)}
    accessibilityLabel=”Mi boton”
    disabled={false}
    color={‘red’}
>
```

El botón que creaste antes ahora podrá usarse para moverse a la pantalla de lista, de modo manual (Figura 2.8.).

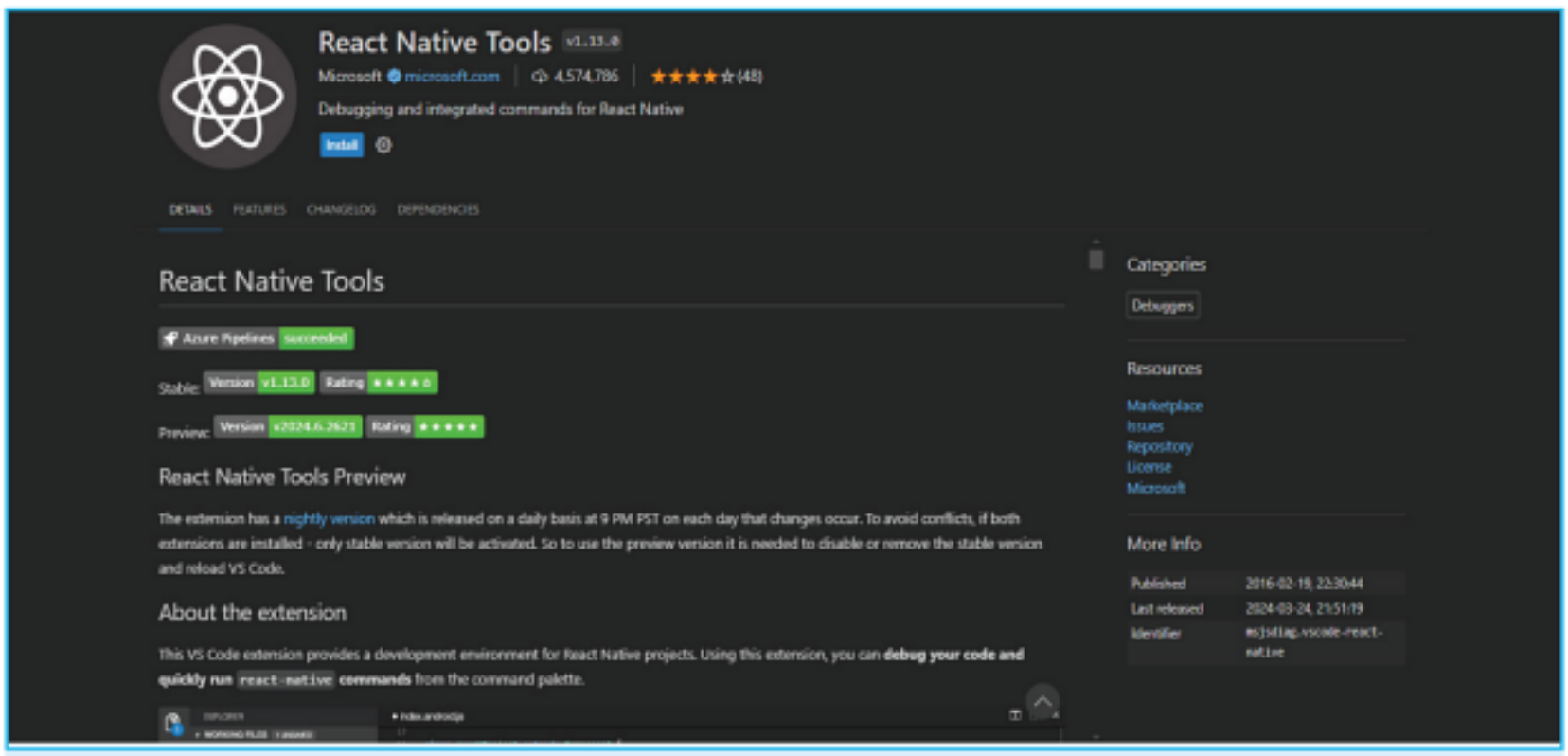


Figura 2.8. *VS Code permite instalar plugins como React Native Tools para mejorar y agilizar la programación en esta plataforma.*

Si bien el elemento **`Button`** es útil para crear un componente que permita realizar una acción al tocarlo, carece de personalización en comparación con otros que ofrece la librería de React Native, con mayores opciones y posibilidades. Por lo tanto, en la pantalla principal, importa el componente **`TouchableOpacity`** y reemplaza el elemento **`Button`** por este nuevo. En su interior coloca un elemento **`Text`** que incluya el **`string Go to List`**, y luego mueve el evento **`onPress`** de **`Button`** al nuevo componente. Debería verse de esta forma:

```
import { TouchableOpacity } from ‹react-native›;

…

return(
…
    <TouchableOpacity
        onPress={() => router.push(‘list’)}
```

2. Estilos

```
          accessibilityLabel=’Go to list’
          style={styles.btnList}
     >
          <Text>
               Go to list
          </Text>
     </TouchableOpacity>
)
```

La principal ventaja de **TouchableOpacity** con respecto a **Button** es la posibilidad de agregar más estilos no disponibles en este último, y poder añadir componentes hijos anidados dentro. Agrega los siguientes estilos:

```
const styles = StyleSheet.create({
     …
     btnList: {
          marginVertical: 15,
          paddingVertical: 15,
          paddingHorizontal: 25,
          backgroundColor: ‘#e5e5e5’,
          borderRadius: 5,
          flexDirection: ‘row’,
          alignItems: ‘center’
     }
});
```

Luego importa el componente **Ionicons** en tu archivo **index.tsx**. Dentro del componente **TouchableOpacity**, agrega un icono de la siguiente manera:

RU

```
<TouchableOpacity
     onPress={() => router.push('list')}
     accessibilityLabel='Go to list'
     style={styles.btnList}
     >
     <Text>
          Go to list
     </Text>
     <Ionicons
          size={16}
          name='arrow-forward'
          color='#000'
     />
</TouchableOpacity>
```

El botón ahora debería funcionar de la misma manera pero, además, tienes la posibilidad de darle estilos personalizados y elementos anidados dentro como un icono.

Actividades

A continuación se presentan las preguntas y los ejercicios que deberías saber responder y resolver para considerar aprendido el capítulo.

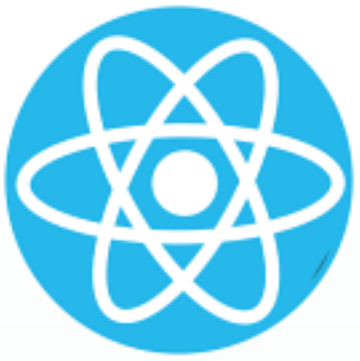
Test de autoevaluación

1. *¿Qué propiedades de Flexbox puedes utilizar dentro de React Native?*
2. *¿Cuál es la orientación de Flex por defecto en React Native?*
3. *¿Qué librerías puedes utilizar para los iconos? ¿Y para la navegación entre pantallas?*
4. *¿Qué hook se emplea para moverse entre pantallas?*

Ejercicios prácticos

1. *Dirígete a la pantalla de inicio y crea otro elemento View.*
2. *Busca una imagen y colócala dentro del proyecto.*
3. *Agrega esta imagen con el elemento Image debajo del botón antes creado.*

Capítulo 03



Datos

En este capítulo, comenzarás a trabajar con datos, creando y usando listas para mostrarlos dentro de la pantalla, y generando las estructuras que te faciliten el desarrollo de la aplicación y su funcionamiento.

Estructuras de datos / 60

Listas y componentes nativos / 66

Peticiones y objetos anidados / 77

Actividades / 82

- Test de autoevaluación / 82
- Ejercicios prácticos / 82

Estructuras de datos

En el mundo del desarrollo, es muy común tener que trabajar con datos que se estructuran de alguna forma particular, ya sea porque provienen de una base de datos, por su validación o por otras razones. En esos casos, la estructura y los atributos que tiene cada objeto ya están definidos y no cambian, incluso, declarando qué propiedades pueden ser opcionales o no. **`TypeScript`** es una gran herramienta para esta tarea. Se trata de un lenguaje de programación considerado un **`superconjunto`** de JavaScript, ya que, básicamente, es el mismo lenguaje, con la misma sintaxis, pero añadiendo el tipado estricto de los datos. Fue desarrollado por Microsoft, y la empresa se encarga de mantenerlo, gracias a lo cual, el editor de código **`Visual Studio Code`** ofrece soporte nativo para él. Es una tecnología de código abierto, es decir que puedes acceder a su repositorio y contribuir al proyecto.

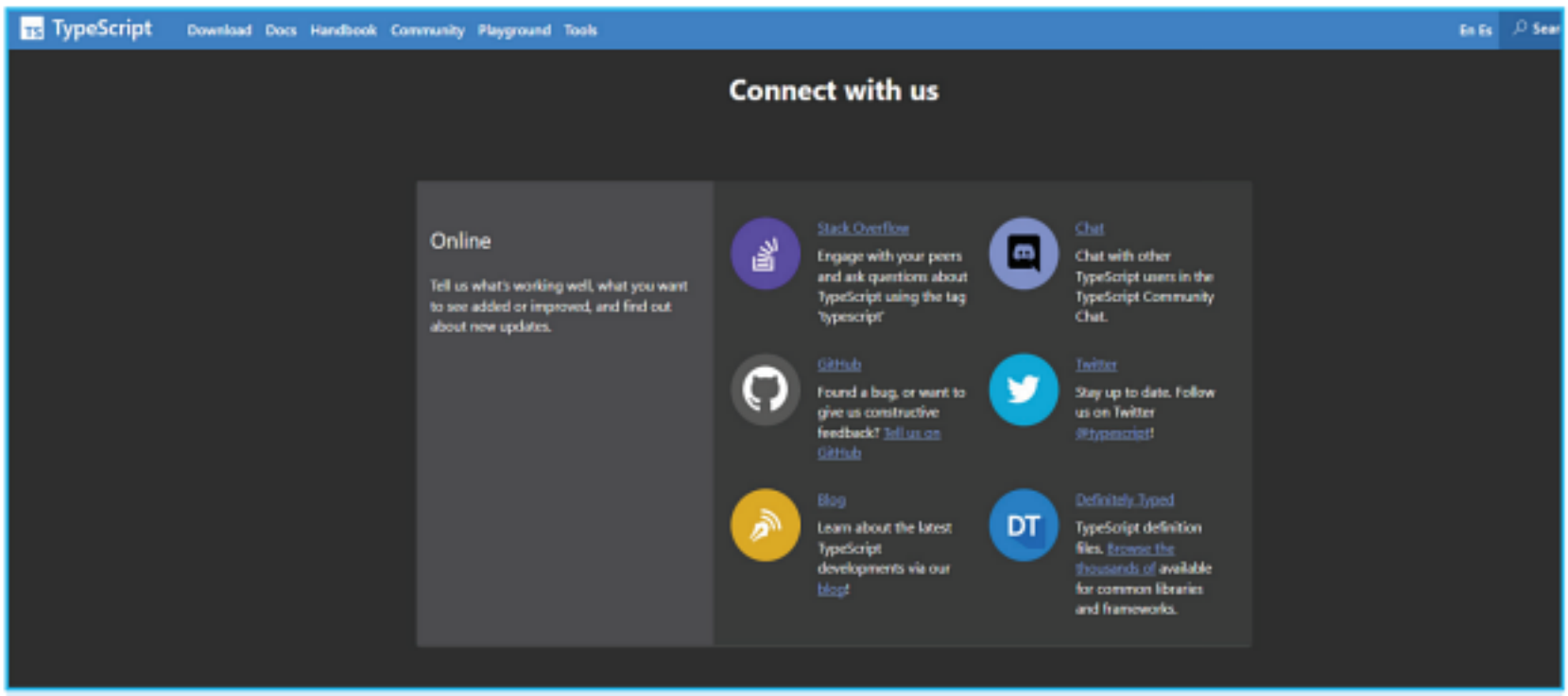


Figura 3.1. *TypeScript es un superconjunto de JavaScript desarrollado y mantenido por Microsoft.*

El tipado estricto en un lenguaje de programación significa que necesitarás definir qué tipo de dato es cada variable que vayas a utilizar en el momento de declararla. Además, implica que una variable

definida como un **`string`** o cadena de caracteres no puede almacenar un número. Si declaras una variable como un dato booleano, no podrás almacenar en ella cadenas de caracteres o números, solo verdadero o falso. Si bien esto parece tedioso o laborioso, evita errores a futuro dado que estos surgirán al momento de compilar; el compilador del lenguaje indicará cuándo estés utilizando un dato erróneo. Además, así se logra optimizar la memoria que los programas utilizan, y se evitan posibles pérdidas de memoria o usos incorrectos que no surgen durante la compilación, sino al ejecutar la aplicación.

Además de los tipos declarados por las librerías o clases customizadas por los desarrolladores, algunos de los tipos que provee TypeScript son **`String`**, **`Number`**, **`Array`**, **`Tuple`**, **`Enum`**, **`Void`**, **`Never`** y **`Any`**.

Un ejemplo de código en TypeScript sería el siguiente:

```
const Title: String = “My App”;

const Stores: String[] = [
    “Store One”,
    “Store Two”,
    “Store Three”
];
```

Como puedes ver, en este ejemplo hay una cadena de caracteres definida como **`Title`** y, antes de declarar su contenido, se indica que esta variable será de tipo **`String`**. Luego, se declara un objeto de tipo lista de **`strings`**, lo cual puedes hacer mediante **`String[]`**, es decir, **`Array`** of **`Strings`**. Si intentas cambiar el contenido de estos datos, por ejemplo, pasando como valor de la primera variable un número o un arreglo de números en el segundo caso, el compilador lanza un error que no permitirá avanzar. Si has trabajado con lenguajes como

C# o **Java**, puedes aprovechar ese conocimiento en este caso ya que también se trata de lenguajes de tipado estricto.

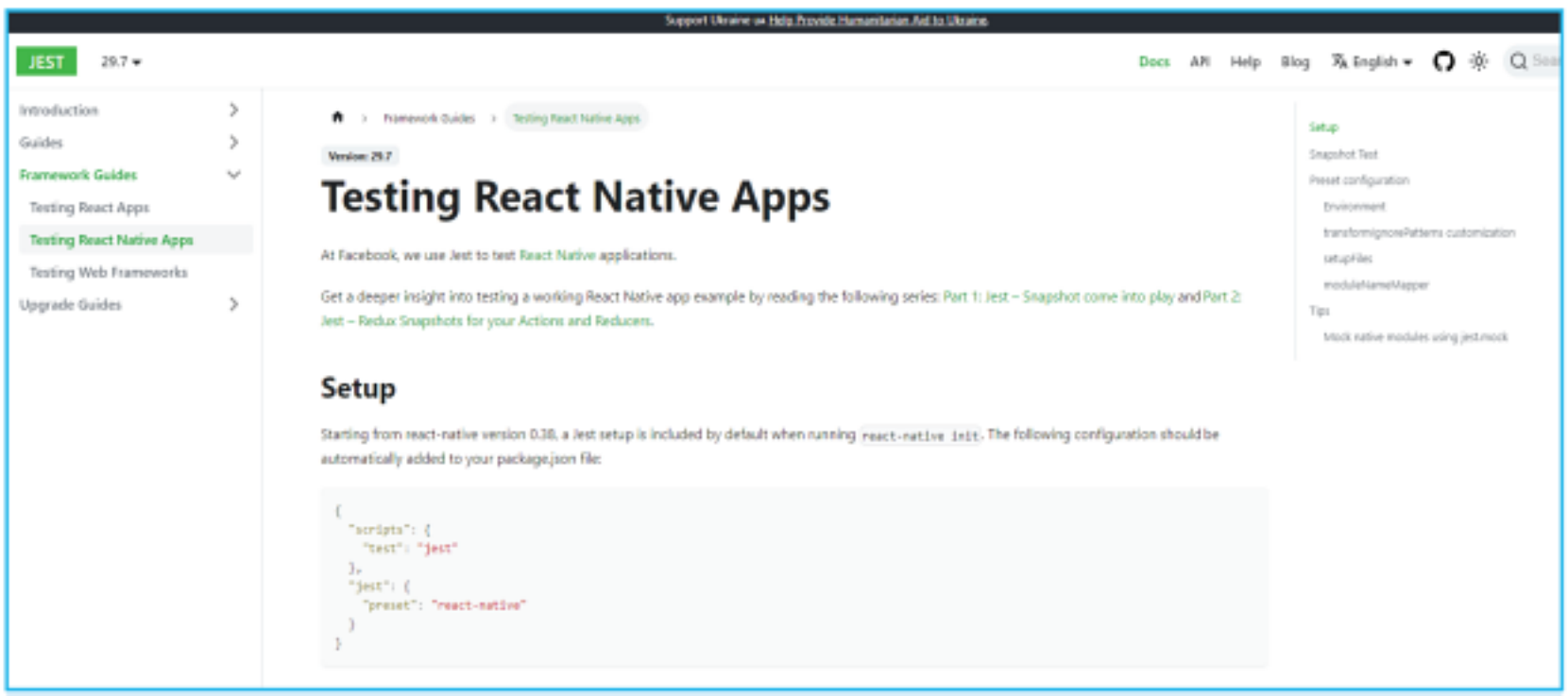


Figura 3.2. *Jest es una librería de testing para aplicaciones desarrolladas en JavaScript.*

Algo interesante de TypeScript es que también puedes definir tus propios tipos declarando clases e interfaces, y utilizarlas en el momento en que necesites crear alguna estructura de datos. Para comenzar, trabajarás con un arreglo de datos estático, y más adelante, lo moverás a una petición HTTP o datos externos. Crea un archivo llamado **data.ts** en tu carpeta **app** y genera una interfaz que te servirá como modelo de un local o negocio físico de tu empresa. La interfaz debería lucir de esta manera:

```
export interface Store {
    id: Number
    title: String
    description: String
    location?: String
    status: Boolean
    employees: Number
```

```
    calification: Number
}
```

TypeScript permite especificar que un dato sea opcional, es decir, que algunas instancias de tus objetos puedan tener o no esa propiedad. En el mismo archivo, crea el siguiente arreglo de datos de tipo **Store**:

```
export const Stores: Store[] =
[
    {
        id: 1,
        title: “Default Store”,
        description: “This is our first store, it is
            located in San Francisco.”,
        location: “Mapple Street 3113”,
        status: true,
        employees: 5,
        calification: 4.5
    },
    {
        id: 2,
        title: “New York Store”,
        description: “This is our New York store.”,
        location: “4th Av. 1234”,
        status: true,
        employees: 2,
        calification: 5
    },
    {
```

3. Datos

```
        id: 3,
        title: “Example Store”,
        description: “This is an example store,
             located in Canada.”,
        location: “Ocean View 9876”,
        status: false,
        employees: 1,
        calification: 4
    },
]
```

Como puedes ver, es posible usar la interfaz **`Store`** como el tipado para los objetos del arreglo. Como mencionamos antes, puedes definir como opcionales algunos de los datos de la clase o interfaz, con lo cual puedes omitir esa propiedad en el momento de utilizarla. En este caso, como creará una lista de locales, puedes declarar la ubicación de algunos de ellos, ya que podría tratarse de comercios electrónicos, sin tiendas físicas, lo que te permitirá agregarlos a la lista sin necesidad de determinar una ubicación. Si intentas borrar la ubicación de alguno de los objetos o agregar uno nuevo sin ubicación, el compilador no indicará ningún problema:

```
{
    id: 4,
    title: “Online Store”,
    description: “This is an online store, is has no
         location.”,
    status: false,
    employees: 1,
    calification: 5
```

```
},
```

Este objeto no lanzará ningún error en la compilación, dado que esta propiedad es opcional a la hora de definirla.

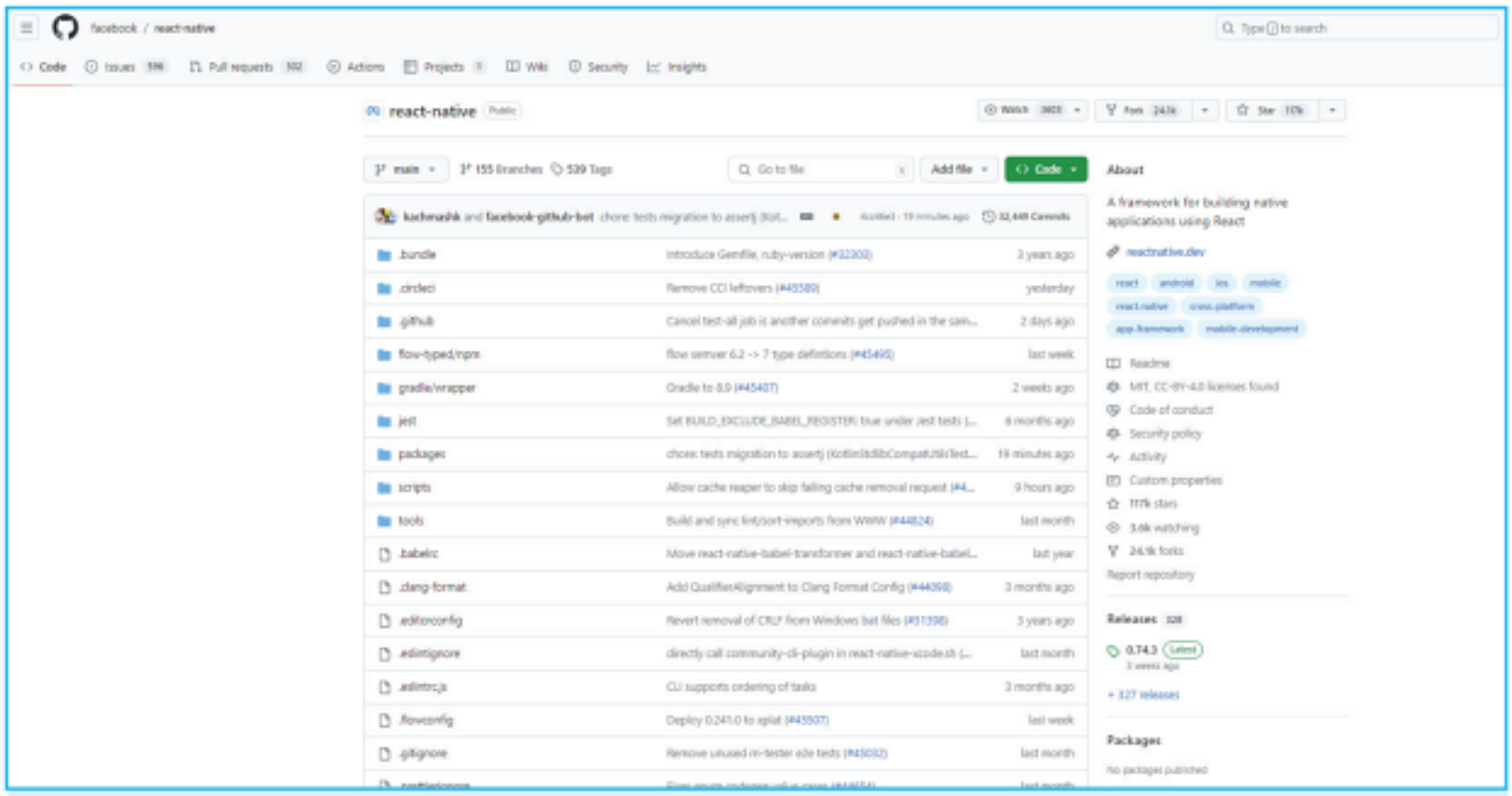


Figura 3.3. *Puedes acceder al repositorio de ReactNative en GitHub y explorar su código fuente o colaborar con el proyecto.*

Ahora, agrega una imagen a la interfaz que has definido y declara su tipo como cadena de caracteres, ya que podrás mostrar la imagen con su atributo SRC, tanto si utilizas un recurso interno como si usas un archivo de tipo **`png`** o **`jpg`** dentro de la aplicación, como un recurso externo. Existen muchos tipos de servicios gratuitos que te permiten acceder a información de prueba como texto o imágenes para desarrollar aplicaciones antes de trabajar con datos reales. En este caso, puedes utilizar placehold.co o betterplaceholder.com, que proveen imágenes gratuitas para hacer las pruebas. Agrega el siguiente atributo a la interfaz **`Store`**:

3. Datos

```
export interface Store {
    id: Number
    title: string
    image: string
...
}
```

Luego actualiza los objetos dentro del arreglo de locales para que todos tengan una imagen, de la siguiente manera:

```
export const Stores: Store[] = [
    {
        id: 1,
        title: “Default Store”,
        image: “https://fakeimg.pl/600x400”,
        description: “This is our first store
            located in San Francisco.”,
        location: “Mapple Street 3113”,
        status: true,
        employees: 5,
        calification: 4.5
    },
    …
}
```

Listas y componentes nativos

Es momento de trabajar con una lista de elementos en la aplicación. Es importante tener en cuenta que, al desarrollar una aplicación para móviles,

esta puede ser vista en distintos dispositivos con pantallas y tamaños muy diversos. Por lo tanto, debes considerar la posibilidad de que el contenido pueda moverse, adaptarse y también admitir **`scroll`** sobre la pantalla para ver todo, en especial, cuando la información puede cambiar.
Actualiza tu lista de importaciones dentro de tu pantalla **`List`**, de la siguiente manera:

```
import {
     StyleSheet,
     View, Text,
     SafeAreaView, FlatList,
     Image }
from ‘react-native’;
```

Has agregado los componentes **`SafeAreaView`**, **`Image`** y **`FlatList`**, los cuales resultan muy útiles para el desarrollo de aplicaciones en React Native. **`Image`**, como puedes imaginar, trabaja de forma muy similar al elemento HTML **`<img>`**, que permite mostrar una imagen en el navegador. Al igual que es ese caso, puedes utilizar un recurso interno al proyecto o importar una URL externa. Si has trabajado con **`NextJS`**, este componente actúa de manera similar al que provee el framework para la web **`Image`**.
El elemento **`FlatList`** es un componente de React Native perfecto para mostrar listas de elementos tanto estáticos como dinámicos. Esta lista permite trabajar con Android e iOS, mostrar contenido en formato horizontal o vertical y, lo más importante, realizar scroll o navegación dentro de la pantalla para acomodar gran cantidad de contenido (Figura 3.4.).

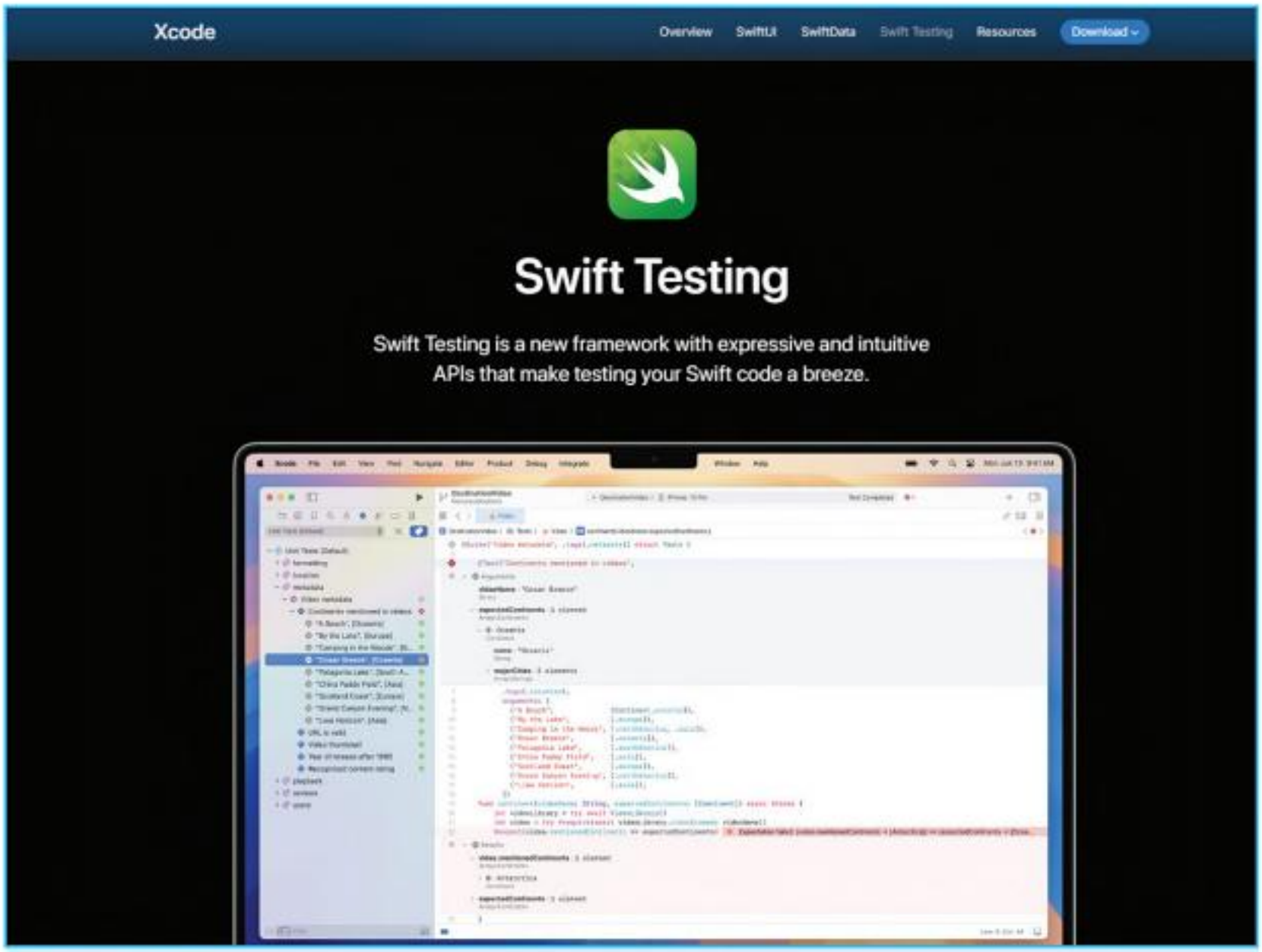


Figura 3.4. *Es muy recomendable testear la aplicación antes de lanzarla y distribuirla para los distintos sistemas operativos.*

Otro elemento fundamental es **`SafeAreaView`**, clave para el desarrollo de aplicaciones en dispositivos móviles. Con la popularización de los teléfonos móviles, han aparecido todo tipo de dispositivos con características distintas, como pantallas con orientación vertical, doble pantalla, pantallas flexibles y también tablets, con lo cual es importante tener un contenedor que permita renderizar elementos dentro de los límites de la pantalla.
Esto es exactamente lo que **`SafeAreaView`** permite hacer, incluso, para dispositivos de tipo iPhone que cuentan con cámaras frontales, ocupando un tamaño considerable en la parte superior (Figura 3.5.).
Es momento de mostrar los datos en pantalla que declaraste dentro del arreglo. Para hacerlo, modifica el contenido de la pantalla **`List`**:

```
export default function ListScreen() {

return (
     <SafeAreaView style={styles.container}>
          <View style={styles.container}>
               <Text style={styles.title}>
                    List of stores
               </Text>
          </View>
     </SafeAreaView>
     );
}
```

Ahora actualiza los estilos de la siguiente manera:

```
const styles = StyleSheet.create({
     title: {
          marginVertical: 15,
          fontSize: 22,
          fontWeight: ‘bold’,
     },
     container: {
          flex: 1,
          justifyContent: ‘center’,
          alignItems: ‘center’,
          backgroundColor: “white”,
     }
});
```

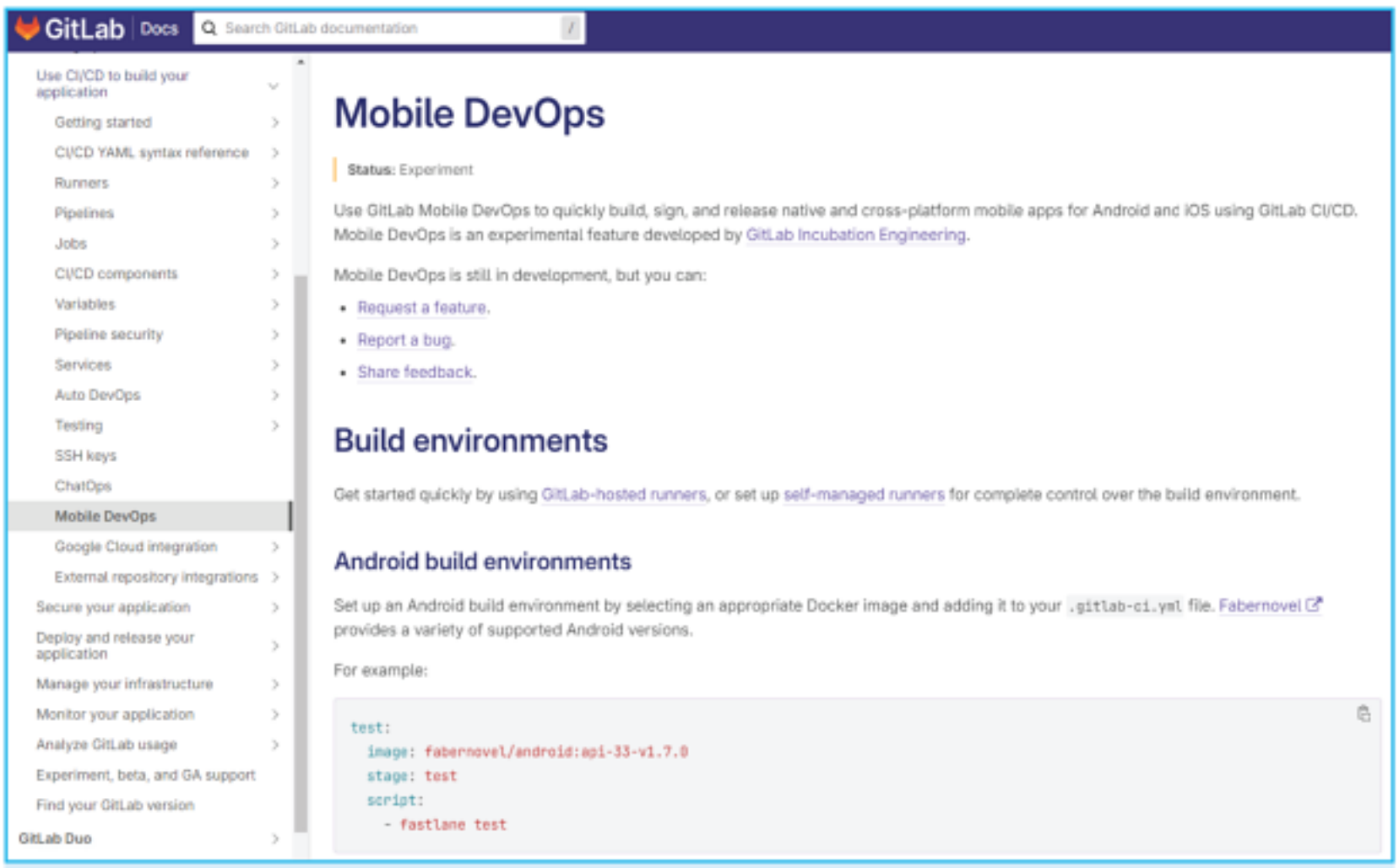


Figura 3.5. *GitLab es una herramienta de repositorios en la nube, que ofrece un gran despliegue de aplicaciones para dispositivos móviles.*

Es momento de utilizar **`FlatList`** para mostrar la lista de elementos. **`FlatList`** pide que utilices un componente para mostrar en cada iteración de los datos que le pases como propiedad. Por ejemplo, si tienes una lista de objetos con cinco elementos, necesitarás declarar un componente que se muestre cinco veces en cada vuelta del bucle. Dentro de la carpeta de componentes, agrega un nuevo directorio con un archivo llamado **`ListItem.tsx`**, y en su interior, coloca el siguiente código:

```
import {
    StyleSheet,
    View,
    Text, Image
} from 'react-native';
import { Store } from '../../app/data';
```

RU

```
const ListItem = (store: Store) => {

return (
     <View style={styles.storeContainer}>
          <View style={styles.imageContainer}>
               <Image
                    source={{ uri: store.image }}
                    style={{
                    width: 100,
                    height: 100,
                    }}
                    resizeMode=’contain’
               />
          </View>
          <View>
               <Text
                    style={styles.storeTitle}>
                         {store.title}
               </Text>
               <Text
                    style={styles.storeDescription}>
                         {store.description}
               </Text>
               <Text style={styles.storeDescription}>
               Employees: {store.employees.toString()}
               </Text>
          </View>
     </View>
     )
};
```

```
export default ListItem;
```

Puedes notar que, además de utilizar la interfaz **`Store`** como declaración en el arreglo de datos, puedes declarar el tipo de propiedad que recibirá este componente de tipo **`Store`**. De esta manera, TypeScript indicará qué propiedades puedes recibir y utilizar desde este objeto.

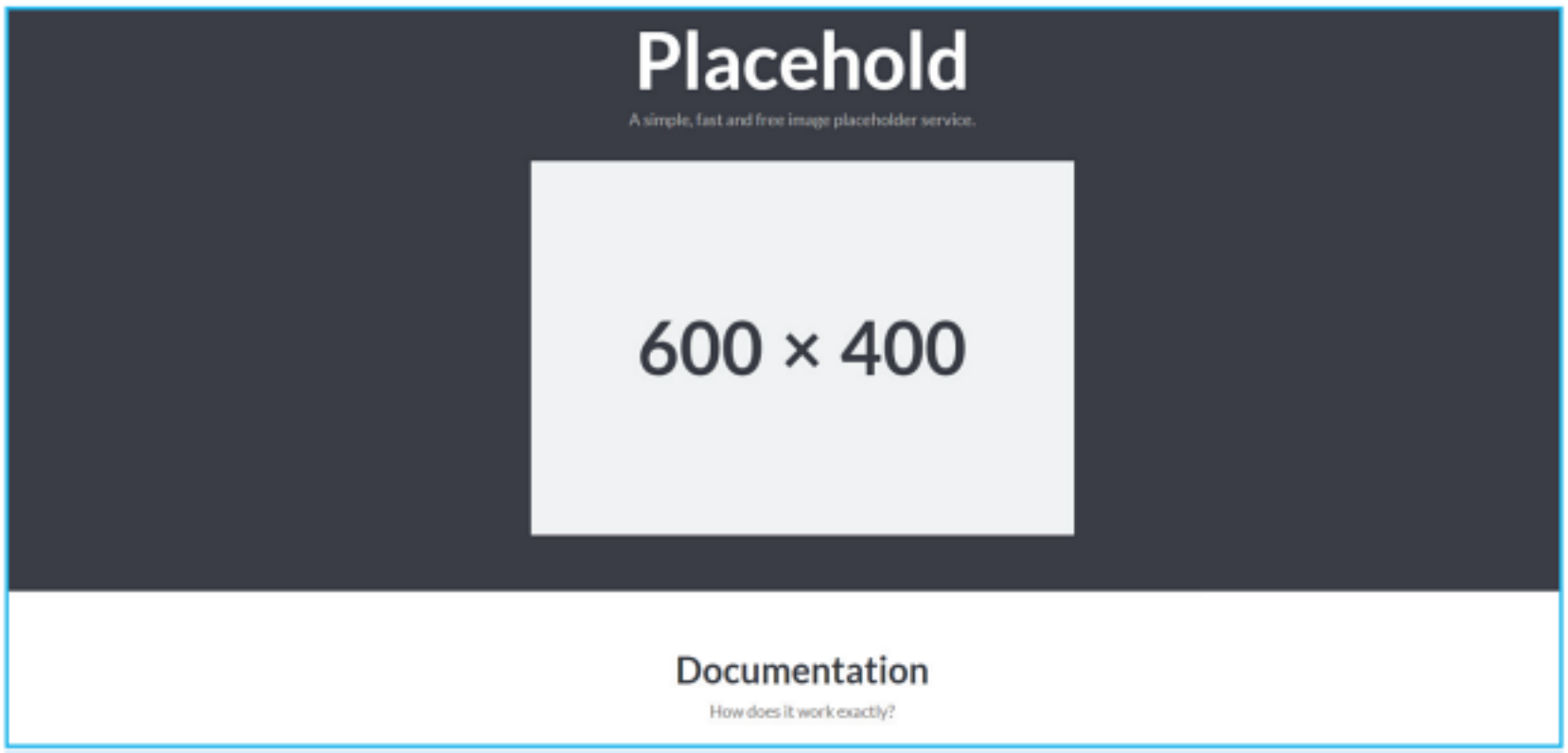


Figura 3.6. *Placehold es un sitio web muy útil para trabajar con imágenes e información de prueba en tus aplicaciones.*

El componente **`Image`**, como puedes ver, funciona de manera similar a **`<img>`**, pero en vez de declarar la propiedad **`src`**, utilizarás el atributo **`source`**, y en su interior, deberás pasar un objeto indicando qué imagen debe mostrarse. En este caso, estarás utilizando una dirección externa, de un recurso en la Web; por ende, tendrás que declarar dentro del objeto un atributo **`uri`**, y allí, pasarle como valor la **`url`** de la imagen. Por otro lado, deberás indicar también un ancho y alto fijos. Es muy recomendable trabajar con el atributo **`resizeMode`**, dado que permite definir cómo se muestra la imagen con respecto al espacio en pantalla. Puedes pensar en este atributo como la propiedad de CSS **`object-fit`** o **`background-size`**,

que permiten ubicar y acomodar la imagen en la pantalla. Además, estarás usando los elementos en cada objeto para mostrar los empleados, la descripción y también un título.
Por último, agrega los siguientes estilos al componente que acabas de crear:

```
const styles = StyleSheet.create({

    storeContainer: {
        marginVertical: 5,
        flex: 1,
        flexDirection: 'row',
        alignItems: 'center',
    },
    imageContainer: {
        paddingHorizontal: 15,
    },
    storeTitle: {
        fontSize: 18,
    },
    storeDescription: {
        fontSize: 12,
    },
});
```

Una vez hecho esto, ya puedes utilizar este componente en toda tu aplicación. Dirígete nuevamente a la pantalla **`List`** y, allí, agrega el siguiente código debajo del título de esta sección:

```
export default function ListScreen() {
```

3. Datos

```
return (
    <SafeAreaView style={styles.container}>
        <View style={styles.container}>
            <Text style={styles.title}>
                List of stores
            </Text>
        </View>

        <FlatList
            data={Stores}
            renderItem={({item}) =>
        <ListItem
            id={item.id}
            title={item.title}
            image={item.image}
            description={item.description}
            status={item.status}
            employees={item.employees}
            calification={item.calification}
        />}
        keyExtractor={item => item.id.toString()}
        />

        </SafeAreaView>
    );
}
```

Ahora se utiliza el componente **`FlatList`** para realizar una lista de las ubicaciones de tus tiendas, y como propiedades del elemento **`ListItem`**, debes enviar las características de un objeto de tipo **`Store`**. También

es importante, como en toda iteración de elementos y al igual que en React, pasar un índice único a cada elemento, que en este caso puede ser el id de cada tienda.

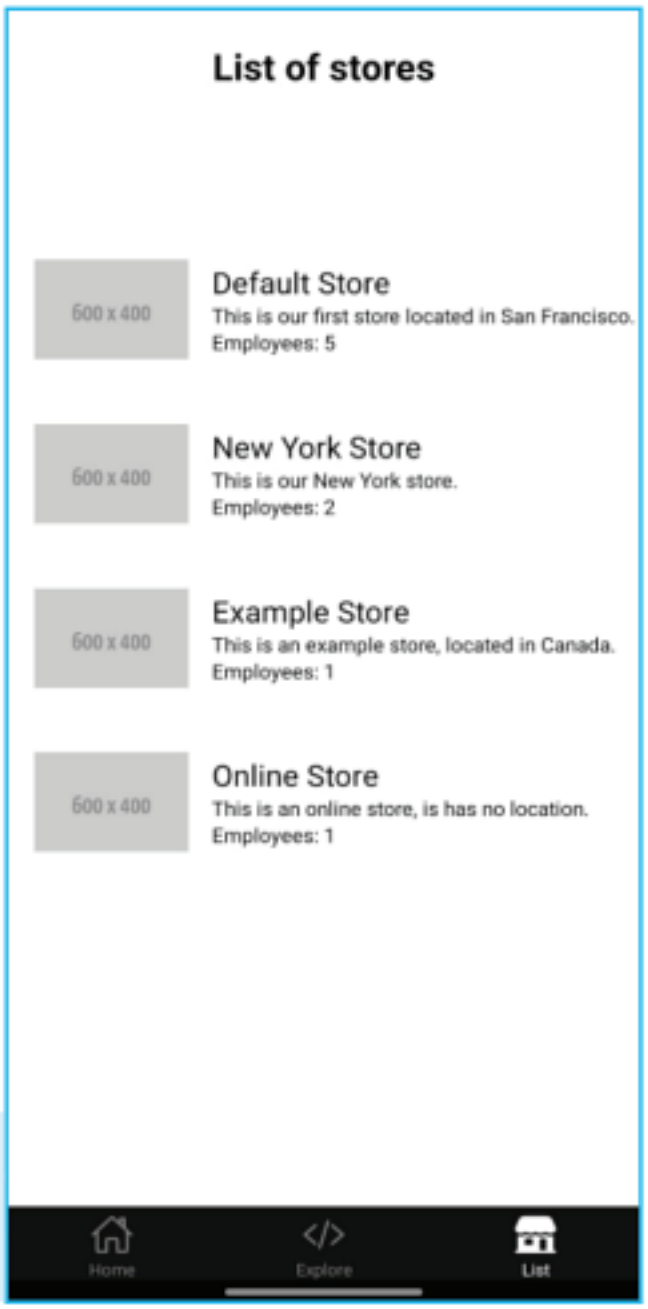


Figura 3.7. *La aplicación se divide en dos partes para el título y cuatro para la lista.*

Dependiendo de tu dispositivo, tal vez necesites agregar más elementos al arreglo de tiendas para poder ver un **`scroll`** en la pantalla. Pero si lo pruebas dentro de un dispositivo más pequeño, es posible que ya se vea una lista con una barra de **`scroll`** lateral. Sin embargo, puedes agregar más elementos a tu arreglo para que la pantalla se ajuste y veas una barra de navegación vertical. En cuanto esto suceda, la lista comenzará a tapar el título de esta sección; esto sucede porque no se ha declarado un comportamiento de tamaño para el elemento de lista con respecto a su contenedor. Para solucionar este problema, agrega el siguiente ajuste al elemento **`FlatList`**:

```
<FlatList
    data={Stores}
```

3. Datos

```
    style={{flex: 1}}
    renderItem={({item}) =>
        <ListItem
            id={item.id}
            title={item.title}
            image={item.image}
            description={item.description}
            status={item.status}
            employees={item.employees}
            calification={item.calification}
        />}
    keyExtractor={item => item.id.toString()}
/>
```

Esto hará que tu pantalla se divida en dos partes: la mitad para el título y una lista más pequeña debajo.

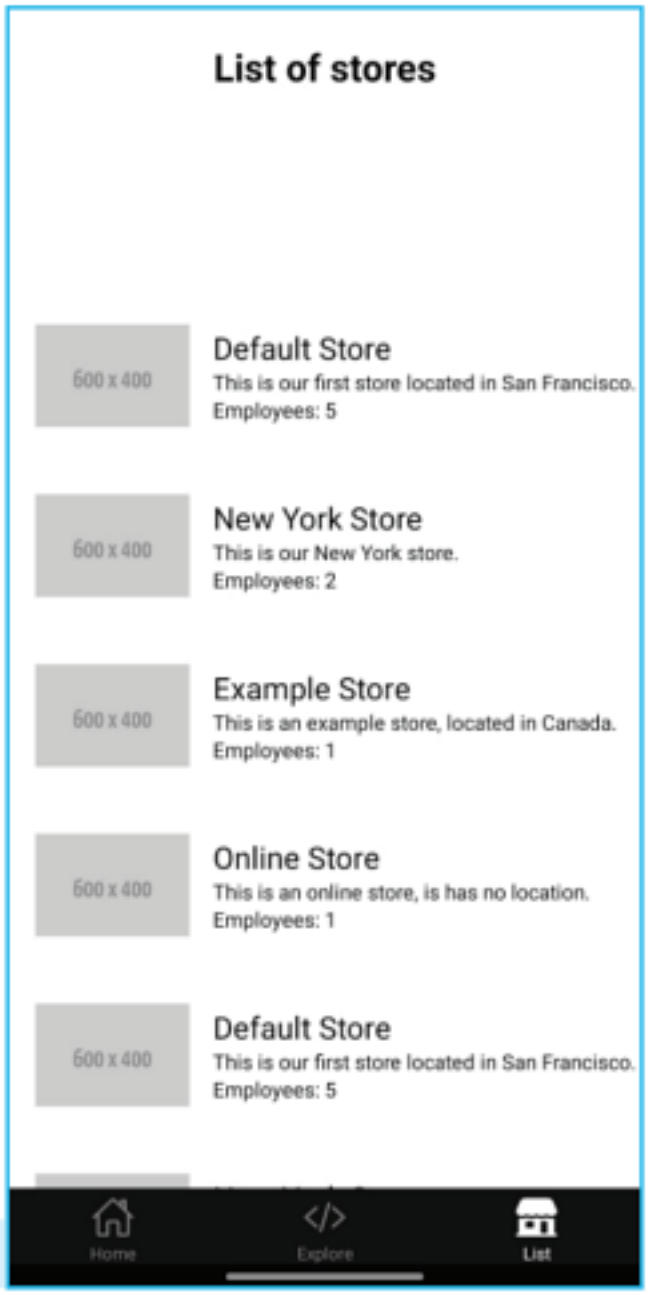


Figura 3.8.

Si deseas modificar este comportamiento, puedes darle más espacio a la lista, agregando un **flex** mayor y realizando un ajuste como el siguiente:

```
<View style={{ flex: 4 }}>
     <FlatList
          data={Stores}
          …
     />
</View>
```

Y luego, asignándole un valor de 2 al atributo Flex que le envíes al contenedor del título.

Peticiones y objetos anidados

Además de poder definir un objeto con propiedades nativas del lenguaje como cadenas de caracteres, valores booleanos, números y arreglos, es posible definir estructuras anidadas dentro de los objetos, por ejemplo, un usuario que, en su interior, almacena la información de la dirección como un objeto con propiedades internas, como número de calle, ciudad, e incluso, un objeto más complejo en su interior con la ubicación geográfica, su latitud y longitud.

La mayoría de las aplicaciones desarrolladas tanto para la Web como para dispositivos móviles no utiliza datos estáticos, como una lista definida dentro de un archivo, sino que suele trabajar con información almacenada de modo seguro en una base de datos. Si estuvieses trabajando con React en la Web, sería imposible conectarte a una base de datos directamente. Como puedes imaginar, tampoco es una opción hacerlo desde una aplicación móvil, ya que los datos deben ser los mismos para cada usuario. Por esta razón, una

arquitectura muy utilizada es separar toda la aplicación del frontend, o la aplicación móvil, del desarrollo para el backend, creado en lenguajes como **Java**, **PHP** o **Python**. Estos trabajan con la información de la base de datos y la exponen de manera segura y estructurada, en formato **JSON** o **XML**, mediante una API y, en muchas ocasiones, solamente tras un proceso de autenticación, solicitando un token o usuario y contraseña.

Ahora, si desearas probar cómo consumir datos complejos desde un backend, realizando peticiones a una API que retorne datos en JSON, necesitarías programar todo un servicio, es decir, crear un servidor con **Node.JS** y **Express**, o con **Laravel**, **Spring** u otro framework para el backend. Esto demandaría muchísimo tiempo y también conocimiento, y un desarrollador tal vez no esté interesado en hacerlo.

Además de servicios que proveen imágenes para probar tus proyectos, existen algunos que permiten realizar una prueba de peticiones a un backend, sin tener que construir tus propios servicios y desarrollar una API completa para tu aplicación móvil, simplemente, probando con data falsa.

Si te diriges a este link, verás que puedes obtener una lista de usuarios ficticia sin tener que programar nada, en formato JSON y listo para ser consumido por tus aplicaciones del frontend, ya sea en React Native, en **React**, **Vue** o en JavaScript y **HTML** nativos.

En este arreglo de objetos, los usuarios poseen en su interior objetos anidados, como su compañía o dirección. En tu aplicación, si deseas consumir estos datos y mostrarlos dentro de la pantalla, tendrás que definir la estructura de los datos como lo hiciste para la lista de tiendas.

Dentro del archivo **data.ts**, agrega la siguiente interfaz:

```
export interface User {
    id: Number
    name: string
```

```
    email: string
    address: Address,
    phone: string,
    website: string,
}
```

Dado que aún no has declarado el tipo **Address**, deberás hacerlo a continuación:

```
export interface Address {
    street: string
    suite: string,
    city: string,
    zipcode: string,
    geo: Geolocalization
}

export interface Geolocalization {
    lat: string
    lng: string
}
```

Puedes utilizar estos tipos de datos para construir un nuevo componente, que al igual que sucede con tus tiendas, deberá mostrar la información de los usuarios. Para lograrlo, crea un componente llamado **UserCard**, y en su interior coloca el siguiente contenido:

```
import {
```

3. Datos

```
    StyleSheet,
    View, Text }
from ‘react-native’;
import { User } from ‘../../app/data’;

const UserCard = (user: User) => {

return (
    <View style={styles.storeContainer}>
        <View>
            <Text style={styles.storeTitle}>
                {user.name}
            </Text>
            <Text style={styles.storeDescription}>
                {user.email}
            </Text>
            <Text
                style={styles.storeDescription}>
                Address: {user.address.city}
            </Text>
        </View>
    </View>
)
};

const styles = StyleSheet.create({

    storeContainer: {
        marginVertical: 5,
        flex: 1,
```

```
            flexDirection: 'row',
            alignItems: 'center',
        },
        imageContainer: {
            paddingHorizontal: 15,
        },
        storeTitle: {
            fontSize: 18,
        },
        storeDescription: {
            fontSize: 12,
        },
});

export default UserCard;
```

Con este componente, puedes mostrar una lista de datos al igual que lo hiciste con la lista de tiendas. En el siguiente capítulo aprenderás cómo crear una lista dinámica con la información de una petición http, así como también realizar pruebas con un emulador.

Actividades

A continuación se presentan las preguntas y los ejercicios que deberías saber responder y resolver para considerar aprendido el capítulo.

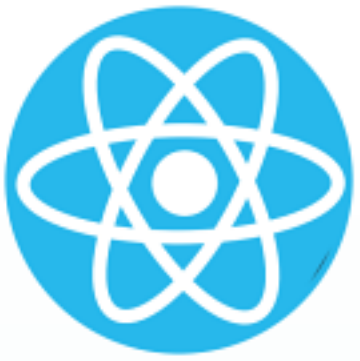
Test de autoevaluación

1. *¿Qué componente se utiliza en React Native para mostrar imágenes?*
2. *¿Qué componente se debe utilizar para asegurar que el contenido de la aplicación se vea completo en pantalla?*
3. *¿Qué ventajas ofrece el uso de TypeScript cuando se crean estructuras de datos?*
4. *¿Para qué se utiliza FlatList?*

Ejercicios prácticos

1. *Dirígete a la pantalla Explorar de tu aplicación y borra el contenido.*
2. *Crea un componente FlatList; este debe utilizar el componente UserCard.*
3. *Aplica la API Fetch para obtener los datos de usuarios y mostrarlos en pantalla.*

Capítulo 04



API externa

Ahora que has aprendido a trabajar con React Native y conoces cómo desarrollar listas de componentes y elementos en arreglos de datos, es momento de aprender a utilizar datos provenientes de un servicio web como una API externa.

Trabajar con los datos del servidor / 84

Formularios en React Native / 94

Actividades / 109

- Test de autoevaluación / 109
- Ejercicios prácticos / 109

Glosario / 110

Trabajar con los datos del servidor

Como puedes imaginar si has trabajado con React en el pasado, la información global de la aplicación, como datos de usuarios o listas de información de tiendas a la cual todo el mundo debe acceder, no tiene que almacenarse en el dispositivo de cada usuario, sino que debe ser accesible desde cualquier aplicación mediante conexiones a un servidor.
Imagina que un usuario necesita registrar sus datos en la aplicación, por ejemplo, cambiar su información personal o enviar consultas a un administrador. En la Web, esto es fácil de lograr mediante el maquetado de un formulario en **HTML** y, por otro lado, un servidor que procesa la información en un script de **PHP**, **Java**, **Ruby** o **Python**. En caso de trabajar con React Native, esto no es posible, dado que en una aplicación para dispositivos móviles solo cuentas con acceso a lo que se guarda en el almacenamiento del teléfono del usuario. Sin embargo, si trabajaste antes con **React**, **Angular** o **Vue.JS**, sabrás también que no es posible guardar datos en el backend, o más bien, dentro de un servidor con estas tecnologías. Una de las razones por las cuales estas tecnologías, entre otras, han resultado muy útiles y se han popularizado tanto en los últimos años, es la separación de las tecnologías del frontend de las del backend. Con el avance del desarrollo web, ha quedado claro que la complejidad de las aplicaciones de uso cotidiano requiere que un programador se especialice en ese ámbito del desarrollo. Un maquetador o desarrollador frontend debe conocer, o al menos debe esperarse que conozca, solamente sobre tecnologías del frontend, como **HTML**, **CSS**, **JavaScript**, frameworks y librerías como **React**, **Svelte** y **Angular**, entre otras; y que maneje estándares como **JSON** y **XML**. Por otro lado, un programador backend solo necesitaría conocer, para un puesto regular, tecnologías del lado del servidor como **PHP**, **Java**, **Python** y **Node.JS**; bases de datos como **MySQL**,

Oracle o **MongoDB**; y librerías y frameworks como **Laravel**, **Spring**, **Django** o **ASP.NET**. Esta separación, si bien significa tener equipos más grandes, permite que cada persona se especialice en un ámbito particular, en vez de tener que conocer todas las tecnologías que un equipo completo maneja. Al especializarse, el desarrollador frontend es capaz de lograr mejores resultados, enfocándose en animaciones, maquetado semántico y optimización de la aplicación; mientras que el programador backend puede optimizar las consultas a la base de datos, mejorar la seguridad y asegurar el correcto desarrollo de su servidor. Esto implica una mejora en la calidad del producto final, aunque conlleva un desarrollo de un equipo mayor y, por ende, una inversión más alta.

React Native, y también otras tecnologías utilizadas para el desarrollo de aplicaciones móviles en JavaScript, orientan su trabajo bajo esta premisa, separando por completo la aplicación de móvil instalable en el dispositivo, de un servidor que contiene una base de datos, con servicios consumibles por la aplicación.

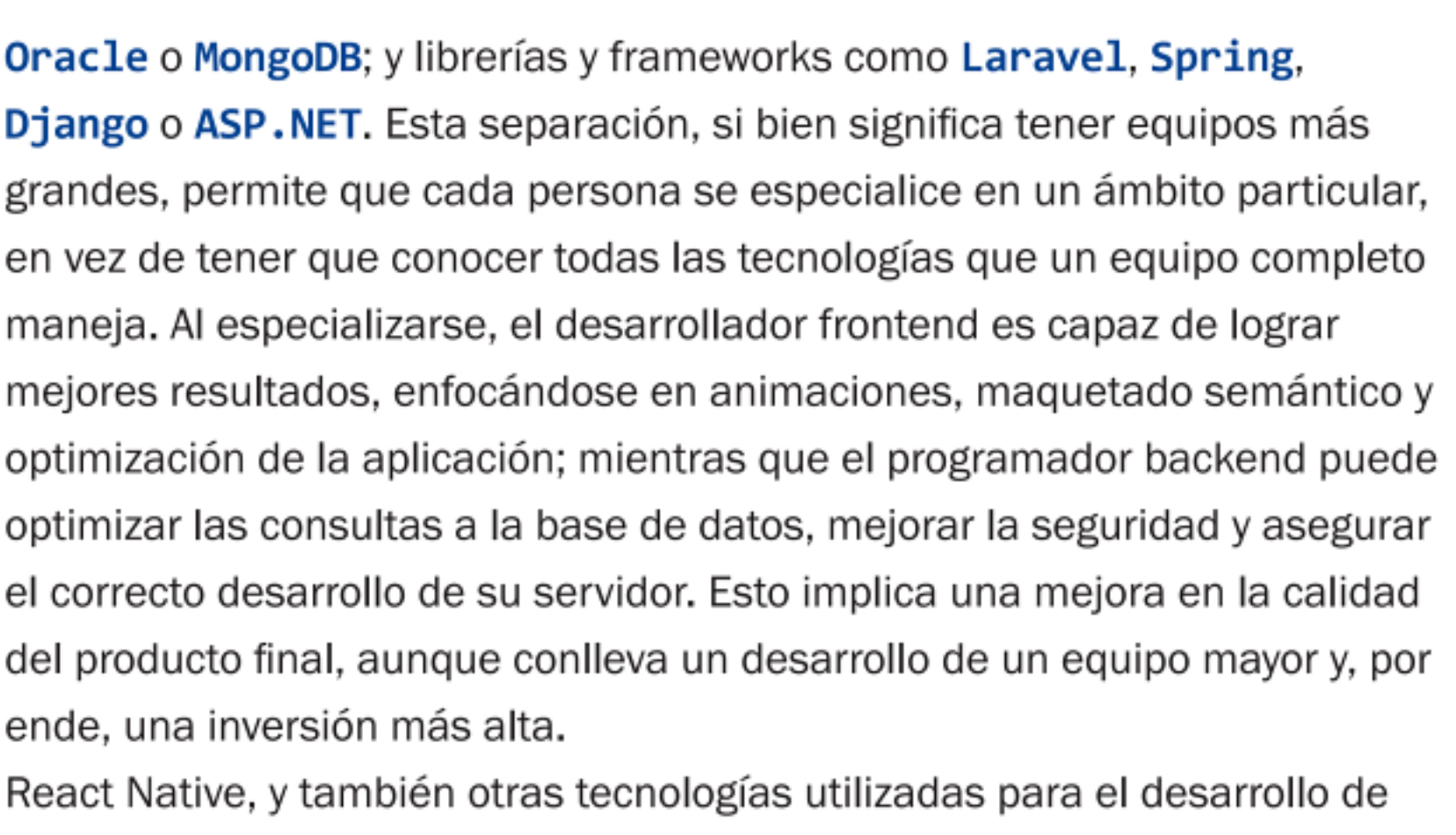


Figura 4.1. *React Native es utilizado ampliamente por muchísimas aplicaciones populares de uso cotidiano.*

Para lograrlo, JavaScript puede utilizar algunas funciones nativas así como también librerías de NPM instalables, que permiten ejecutar consultas a servicios web de manera asíncrona. Esto significa que el código no es bloqueante, es decir que las respuestas de un servicio externo no bloquearán el uso de la aplicación, sino que puedes ejecutar código simultáneamente mientras el servidor procesa las peticiones. Como se mencionó en capítulos anteriores, para realizar peticiones a servicios web mientras pruebas la aplicación, no es preciso desarrollar todo un servidor, sino que puedes aprovechar algunos de los recursos externos que existen dentro de la Web. Esto es tanto para realizar peticiones de tipo HTTP que consuman datos, como para enviar datos y recibir una respuesta de éxito o fracaso en la aplicación. En el capítulo anterior, explicamos cómo puedes utilizar un servicio como **`JSON Placeholder`**, realizando peticiones a sus servicios para obtener información falsa. Por ejemplo, si accedes en tu navegador a la dirección ingresando a este link, verás una lista de usuarios en formato JSON.

Para acceder a esta información en tiempo real, puede utilizar la API **`Fetch`** de JavaScript en tu aplicación y realizar una petición a este servicio. Este tipo de acciones requiere el uso del hook de **`useEffect()`** en tu componente, que funciona igual que en React, al desarrollar una aplicación web. En primer lugar, dirígete al componente **`Explore.tsx`** y allí crea una función llamada **`getUsers()`**. Esta debe encargarse de realizar una petición al servicio antes mencionado, y luego procesar los datos que están en formato JSON a un objeto de JavaScript:

```
const getUsers = () => {
    fetch(
    `https://jsonplaceholder.typicode.com/users`);
}
```

Como puedes ver, la función **`fetch`** toma como parámetro la URL del servicio. También es importante aclarar que esta función es asíncrona, con lo cual debes esperar la respuesta del servicio para mostrar su contenido. Es posible declarar esta función como asíncrona para utilizar el operador **`await`** en su interior:

```
const getUsers = async () => {
    fetch(    `https://jsonplaceholder.typicode.com/
users`)
}
```

Para parsear el contenido de la respuesta a un objeto de JavaScript, puedes utilizar la instrucción **`then()`** y, en su interior, colocar el siguiente código:

```
const getUsers = async () => {
    fetch(`https://jsonplaceholder.typicode.com/users`)
    .then(response => response.json())
    .then(
        res => {
            console.log(res);
        },
        err => {
            console.log(err);
        }
    );
}
```

Como puedes notar, en este ejemplo se parsea la información y se muestra en la consola. Dado que en el desarrollo de aplicaciones nativas no hay una terminal en el navegador, puedes ver el resultado en la consola integrada

en el editor que utilizaste para iniciar la aplicación. Ya que necesitarás mostrar este contenido en la pantalla del dispositivo, debes guardar los datos dentro de un objeto accesible para todo el componente. Para hacerlo, puedes utilizar el hook de **useState**, de esta manera:

```
import { useEffect, useState } from 'react';

export default function TabTwoScreen() {

    const [users, setUsers] = useState<User[]>();

    return (
        ...
    )
}
```

Una vez hecho esto, podrás guardar el contenido de la respuesta en el objeto **users**, que has definido de tipo arreglo de objetos de la interfaz **User**.

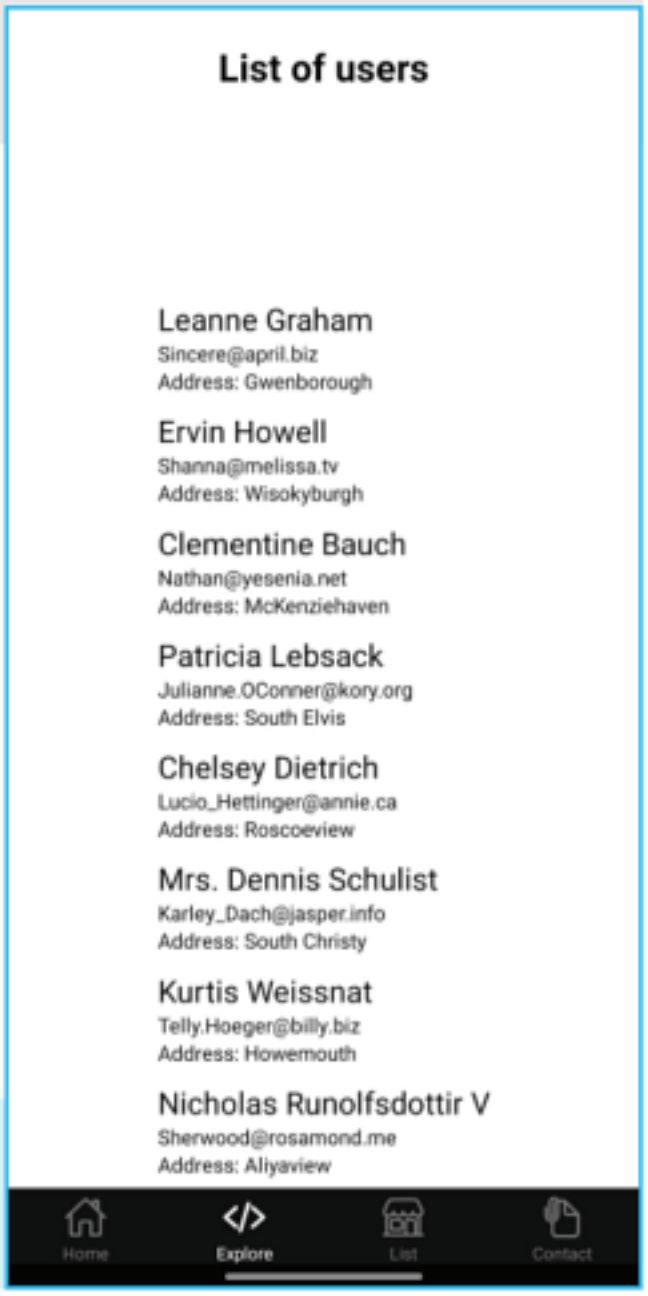


Figura 4.2. *Expo permite ver el resultado de un programa en tiempo real con recarga luego de cada guardado.*

A continuación, guarda el resultado de la respuesta en este objeto:

```
import { useEffect, useState } from ‘react’;
import { User } from ‘../data’;
import UserCard from ‘@/components/UserCard’;

export default function TabTwoScreen() {

    const [users, setUsers] = useState<User[]>();

    const getUsers = async () => {
    fetch(`https://jsonplaceholder.typicode.com/users`)
    .then(response => response.json())
    .then(
         res => {
              console.log(res);
              setUsers(res);
         },
         err => {
              console.log(err);
         }
    );
}

return(
         …
    )

}
```

Ya puedes llamar a esta función dentro del hook **useEffect**:

```
export default function TabTwoScreen() {

    const [users, setUsers] = useState<User[]>();

    useEffect(() => {
        getUsers();
    }, []);

}
```

Dentro de la instrucción **return()**,utiliza el objeto **users** de esta forma:

```
<View style={{ flex: 4 }}>
{
    users
    &&
    <FlatList
        data={users}
        renderItem={({ item }) =>
            <UserCard
                id={item.id}
                email={item.email}
                name={item.name}
                address={item.address}
                phone={''}
                website={''}
            />
```

```
            }
                keyExtractor={
                    item => item.id.toString()}
        />
    }
    </View>
```

Se mostrará una lista de datos de los usuarios en la pantalla de la aplicación. Estos datos, ahora, en vez de provenir de la lista de objetos en JSON que declaraste dentro de la aplicación, se buscan desde el servicio web que provee **`JSONPlaceholder`**. Un punto importante es que, dado que ya no se trata de datos que estás declarando dentro de tu aplicación, estás frente a información que depende de un servidor. Por lo tanto, considera que este servicio puede fallar o devolver un error, o que el dispositivo puede no contar con conexión a Internet. Por esta razón, es importante capturar las excepciones o errores que pueden surgir en el momento de realizar las peticiones.

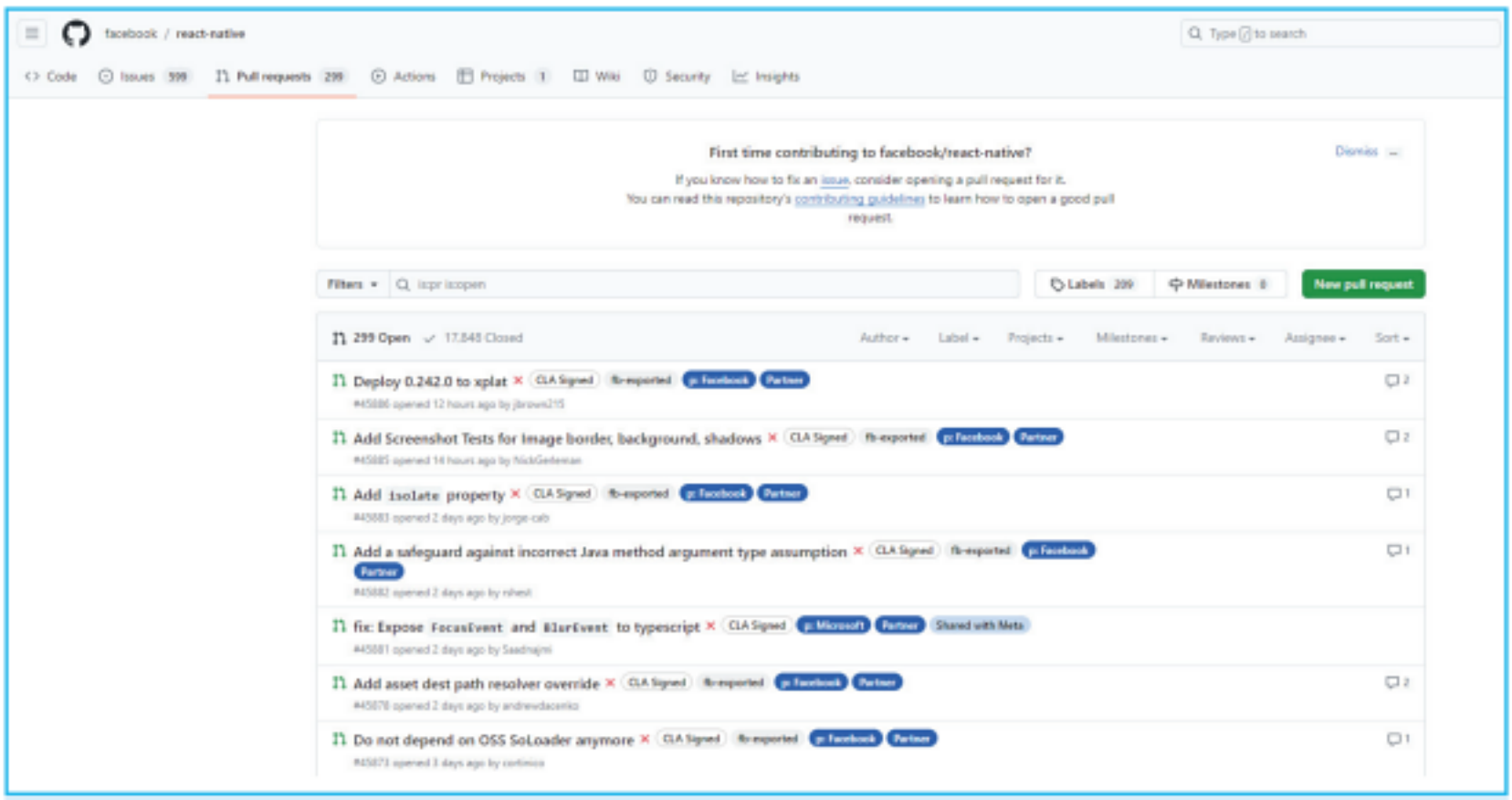


Figura 4.3. *Si lo deseas, puedes contribuir con React Native por medio de ideas, resolución de problemas o mejoras del código fuente.*

4. API externa

Modifica la función que realiza la petición **HTTP** para que cambie el estado de una nueva variable, llamada **error**, al valor verdadero en caso de que surja un error al obtener la respuesta:

```
const [error, setError] = useState(false);
const [users, setUsers] = useState<User[]>();

const getUsers = async () => {
    fetch(`https://jsonplaceholder.typicode.com/users`)
    .then(response => response.json())
    .then(
        res => {
            console.log(res);
            setUsers(res);
        },
        err => {
            console.log(err);
            setError(true);
        }
    );
}
```

Luego, puedes utilizar un ternario para verificar el valor de la variable **error**, y mostrar un mensaje en cada caso:

```
<View style={{ flex: 4 }}>
    {
        error
        ?
```

```
          <Text>
               Hubo un error al procesar la
petición.
          </Text>
:
users
?
          <FlatList
               data={users}
               renderItem={({ item }) =>
               <UserCard
                    id={item.id}
                    email={item.email}
                    name={item.name}
                    address={item.address}
                    phone={‘’}
                    website={‘’}
               />
               }
               keyExtractor={item =>
                    item.id.toString()}
          />
     :
               <Text>
                    No se encontraron usuarios en
               el sistema
               </Text>
     }
</View>
```

De esta manera, solo se mostrarán los usuarios en caso de que haya una respuesta satisfactoria en el servidor.

Formularios en React Native

Además de realizar peticiones en las que obtengas datos desde una respuesta HTTP (lo que usualmente se realiza mediante una petición GET), en muchísimas ocasiones también es necesario hacer peticiones que envían datos al servidor, no solo que los consumen. En las aplicaciones móviles, es muy común tener que hacer envíos de información, como contactar a los administradores, realizar cambios en la información personal del usuario, hacer compras o mandar datos de cualquier tipo. Como ya sabes, en la Web, esto se puede llevar a cabo de manera muy simple, mediante un formulario. Pero en las aplicaciones nativas, tendrás que conectar los campos que el usuario complete con la petición, que puede ser de tipo **POST**, **PUT** o cualquier otro verbo **HTTP** que se encarga de realizarla.

Para comenzar, utilizaremos otra librería HTTP que te permitirá enviar los datos a un servidor. Instala **axios**, un paquete **npm** que se utiliza para el envío de datos por medio de peticiones HTTP en aplicaciones JavaScript. Solo tienes que correr el comando dentro de tu proyecto **npm install axios**, y ya podrás utilizar esta librería en toda la aplicación. Una vez hecho esto, crea una nueva pantalla con su **tab** correspondiente, llamada Form. tsx; deberá lucir de la siguiente manera:

```
import { StyleSheet, View, Text, TouchableOpacity } from
‹react-native›;
import { Ionicons } from '@expo/vector-icons';

export default function FormScreen() {
```

RU

```
return (
<View style={styles.container}>
     <Text style={styles.title}>
          Contactanos
     </Text>

     <TouchableOpacity
          onPress={() => console.log('sending')}
          accessibilityLabel='Go to list'
          style={styles.btnList}
     >
          <Text>
               Enviar{" "}
          </Text>
          <Ionicons
                    size={16}
                    name='document-attach'
                    color='#000'
               />
          </TouchableOpacity>
     </View>
  );
}
const styles = StyleSheet.create({
     title: {
          marginVertical: 15,
     },
     container: {
          flex: 1,
          justifyContent: 'center',
```

```
          alignItems: ‘center’,
          backgroundColor: “white”,
     },
     btnList: {
          marginVertical: 15,
          paddingVertical: 15,
          paddingHorizontal: 25,
          backgroundColor: ‘#e5e5e5’,
          borderRadius: 5,
          flexDirection: ‘row’,
          alignItems: ‘center’
     }
});
```

Agrega al layout de la aplicación el siguiente componente **Tab** al final:

```
return (
     <Tabs
          screenOptions={{
               tabBarActiveTintColor:
               Colors[colorScheme ?? ‘light’].tint,
          headerShown: false,
     }}>

     …
          <Tabs.Screen
               name=”form”
               options={{
               title: ‘Contact’,
```

```
                tabBarIcon: ({ color, focused }) => (
                    <TabBarIcon
                        name={focused ?
                        'document-attach'
                        : 'document-attach-outline'}
                        color={color} />
                    ),
                }}
        />
    </Tabs>
)
```

Ya tienes una nueva pantalla, y la aplicación en el emulador debería poder mostrarla accesible desde el menú inferior. Puedes comenzar a trabajar con el formulario de contacto.



Figura 4.4. *Axios es una librería open source para las peticiones HTTP en aplicaciones JavaScript.*

Dentro de la Web, tanto si trabajas con HTML y JavaScript plano, sin ningún framework o librería, como si lo haces con React o Angular, puedes recurrir a las etiquetas **`input`** y **`textarea`** tradicionales para obtener datos del usuario.
Como puedes imaginar, dentro de React Native estas etiquetas no existen, pero el framework incluye algunas etiquetas con funcionamiento similar que puedes aprovechar. En este caso, la más común es **`TextInput`**, una etiqueta similar a un input tradicional, que se usa para definir distintos tipos de entrada del usuario. La etiqueta **`TextInput`** permite, por defecto, definir una entrada de texto estándar de una línea, donde el usuario puede escribir caracteres sin limitaciones. Puedes asociar este componente, al igual que en React, a una variable de estado, para almacenar en memoria su contenido. También puedes definir un **`placeholder`** como en la Web, configurar el contenido como texto plano o solo numérico, definir el tipo de teclado que el usuario verá dependiendo de su dispositivo (solo números o el teclado estándar), entre muchas otras opciones, y, como todos los demás componentes, declarar un objeto con estilos.

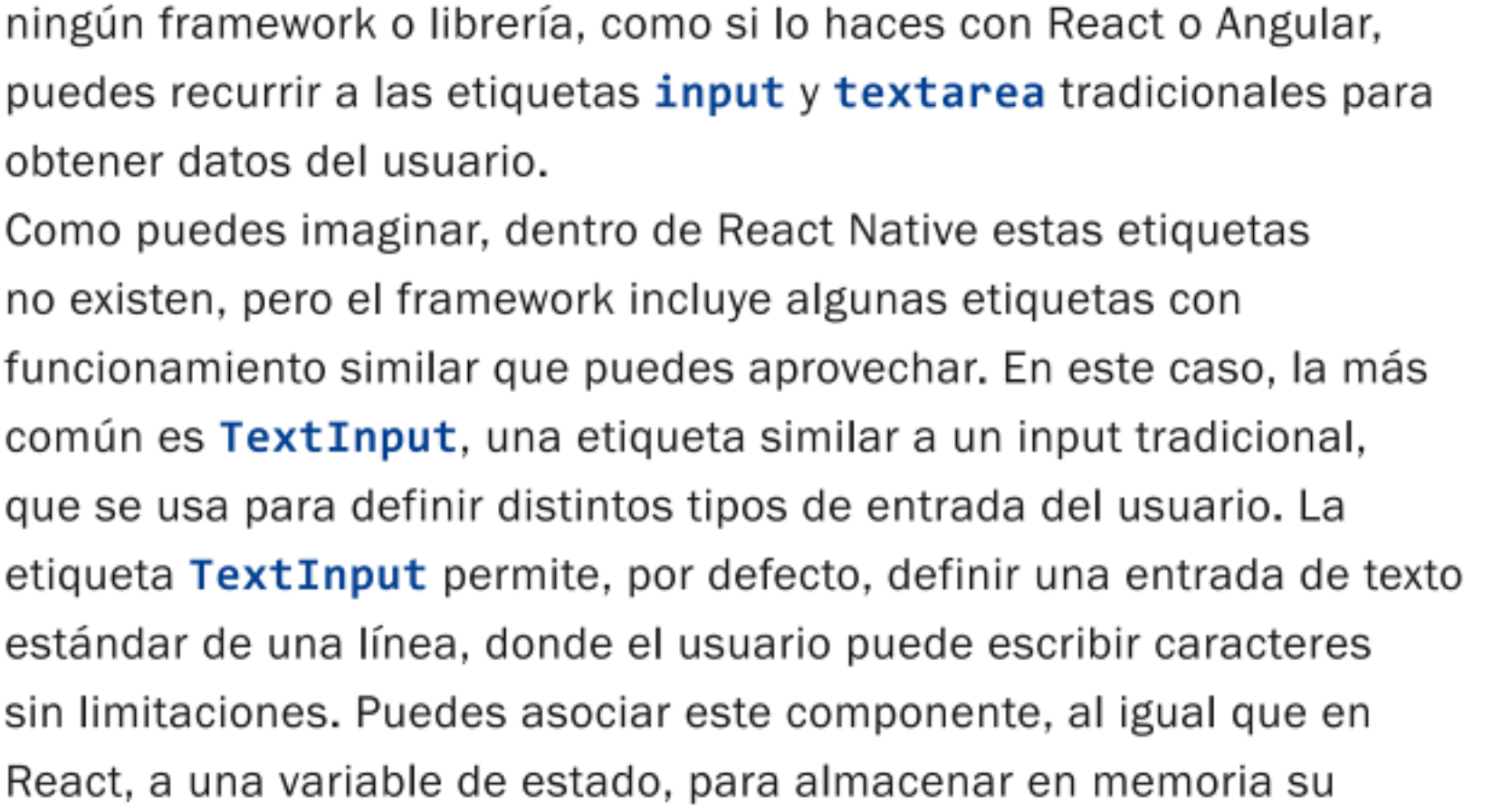


Figura 4.5. *Las aplicaciones de Meta están desarrolladas en React Native.*

Para comenzar, agrega estos objetos a tu hoja de estilos en la página en la que estás trabajando:

```
inputContainer: {
    marginVertical: 10,
},
input: {
    height: 40,
    padding: 10,
    borderBottomColor: ‘#f6f6f6’,
    borderBottomWidth: 2
},
```

Agrega el **import** correspondiente desde **react-native**, y configura el hook de estado para una variable de tipo texto llamada **name**, de esta manera:

```
import {
    StyleSheet, View,
    Text, TouchableOpacity,
    TextInput } from ‘react-native’;
import { Ionicons } from ‘@expo/vector-icons’;
import { useState } from ‘react’;

export default function FormScreen() {

    const [name, setName] = useState(“”);
    …
}
```

Por último, utiliza el componente **`TextInput`**, declarándole los estilos que definiste anteriormente. Por medio de las props **`value`** y **`onChangeText`**, define qué valor debe almacenar el componente para que luego pueda usarse en una petición HTTP o en otro evento. También puedes definir un **`placeholder`** mediante esa prop. Para declarar un tipo de teclado en particular, usa **`keyboardType`** como propiedad del componente, de esta forma:

```
<View style={styles.inputContainer}>
    <TextInput
        style={styles.input}
        onChangeText={setName}
        value={name}
        placeholder=”Su nombre”
        keyboardType=”default”
    />
</View>
```

A continuación, es necesario comprobar que los datos que se guardan en el **`TextInput`** estén correctamente ligados a la variable de estado antes definida. Para esto, puedes aprovechar el botón que creaste antes y mostrar por consola el contenido de la variable **`name`**. En primer lugar, declara una nueva función llamada **`submit()`**, que debe verse de esta manera:

```
const [name, setName] = useState(«»);

const submit = () => {
    console.log(name);
}
```

Luego, modifica el botón antes creado cambiando su prop **onPress** para que llame a la función **submit** cuando alguien lo presione:

```
<TouchableOpacity
    onPress={() => submit()}
    accessibilityLabel='Go to list'
    style={styles.btnList}
>
    <Text>
        Enviar{" "}
    </Text>
    <Ionicons
        size={16}
        name='document-attach'
        color='#000'
    />
</TouchableOpacity>
```

En la terminal del editor de código, deberías ver impreso lo que hayas escrito dentro del **TextInput**. Ahora que lograste agregar un campo para el nombre, comienza a modificar el formulario y agrega múltiples campos dependiendo de tus necesidades. Dentro de los formularios, tanto para la Web como para cualquier otra aplicación, es muy frecuente encontrar campos de texto con varias líneas, conocidos en la Web como **textarea**, donde el usuario puede escribir un texto mucho más largo que un nombre o correo electrónico. React Native provee al componente **TextInput** de varias props que permiten adaptarlo a las necesidades del usuario, convirtiendo el campo simple en un campo de texto mucho más amplio. Para esto, es necesario agregar las props **multiline** y **numberOfLines**, que permiten disponer de un área de texto mayor:

```
<View style={styles.inputContainer}>
    <TextInput
        multiline
        numberOfLines={5}
        style={styles.input}
        onChangeText={setMessage}
        value={message}
        placeholder=”Su mensaje”
        keyboardType=”default”
    />
</View>
```

Otra propiedad interesante es **`editable`**, que toma como parámetro un valor booleano, es decir, verdadero o falso, y te permite definir si un campo de texto puede ser o no editado por el usuario de la aplicación. También existen otras propiedades, como **`autoCapitalize`**, que se usa para declarar si un campo de texto, por defecto, pone en mayúsculas la primera letra que ingrese el usuario.

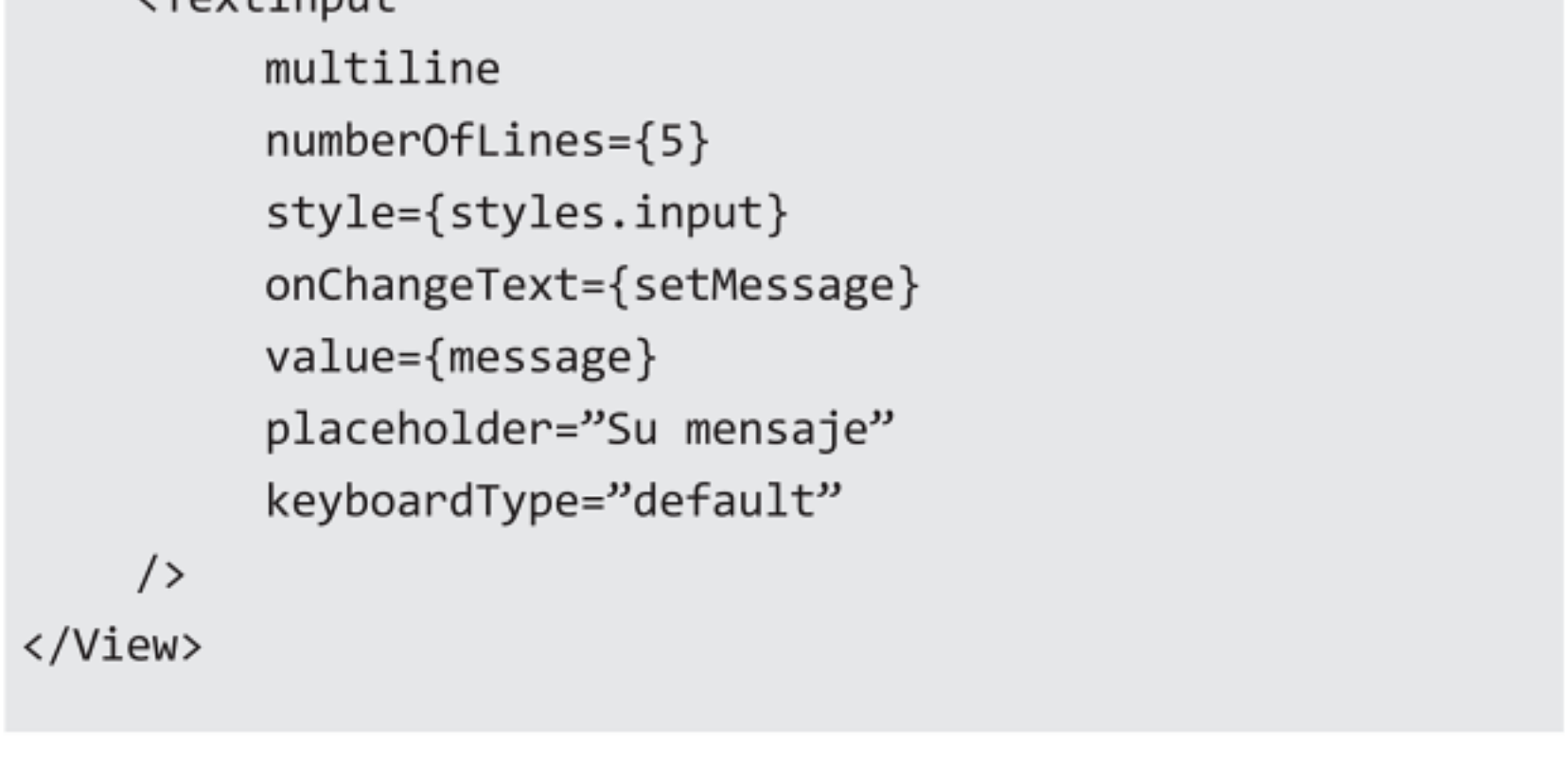


Figura 4.6. *Visual Studio Code brinda sugerencias de las propiedades de los componentes y también de los valores para cada una.*

También puedes declarar la propiedad **autoFocus**, mediante la cual, cuando se monta el componente, automáticamente el foco de la aplicación pasa a este campo de texto, como si el usuario la hubiese presionado, y abre el teclado del dispositivo.
El evento **onFocus** es otra propiedad interesante que permite cambiar el estado o realizar una acción cuando el foco pasa a estar sobre este componente, es decir, cuando lo presiona el usuario.
También puedes definir la propiedad **defaultValue**, que coloca un valor por defecto que se eliminará cuando el usuario comience a escribir. Para validar el máximo de caracteres que un usuario puede agregar a los campos de texto, usa la propiedad **maxLength**, de la siguiente manera:

```
<TextInput
      style={styles.input}
      onChangeText={setName}
      value={name}
      placeholder=”Su nombre”
      keyboardType=”default”
      maxLength={60}
/>
```

Antes de utilizar estos o cualquier otro tipo de evento o propiedad, es importante revisar qué tipo de comportamiento se espera en un dispositivo con Android e iOS. Esto se debe a que, al trabajar con dos sistemas operativos en paralelo, algunas propiedades y eventos pueden variar o no estar disponibles para alguno de los dos, incluso, en distintas versiones de React Native o del sistema operativo (Figura 4.7.).

4. API externa

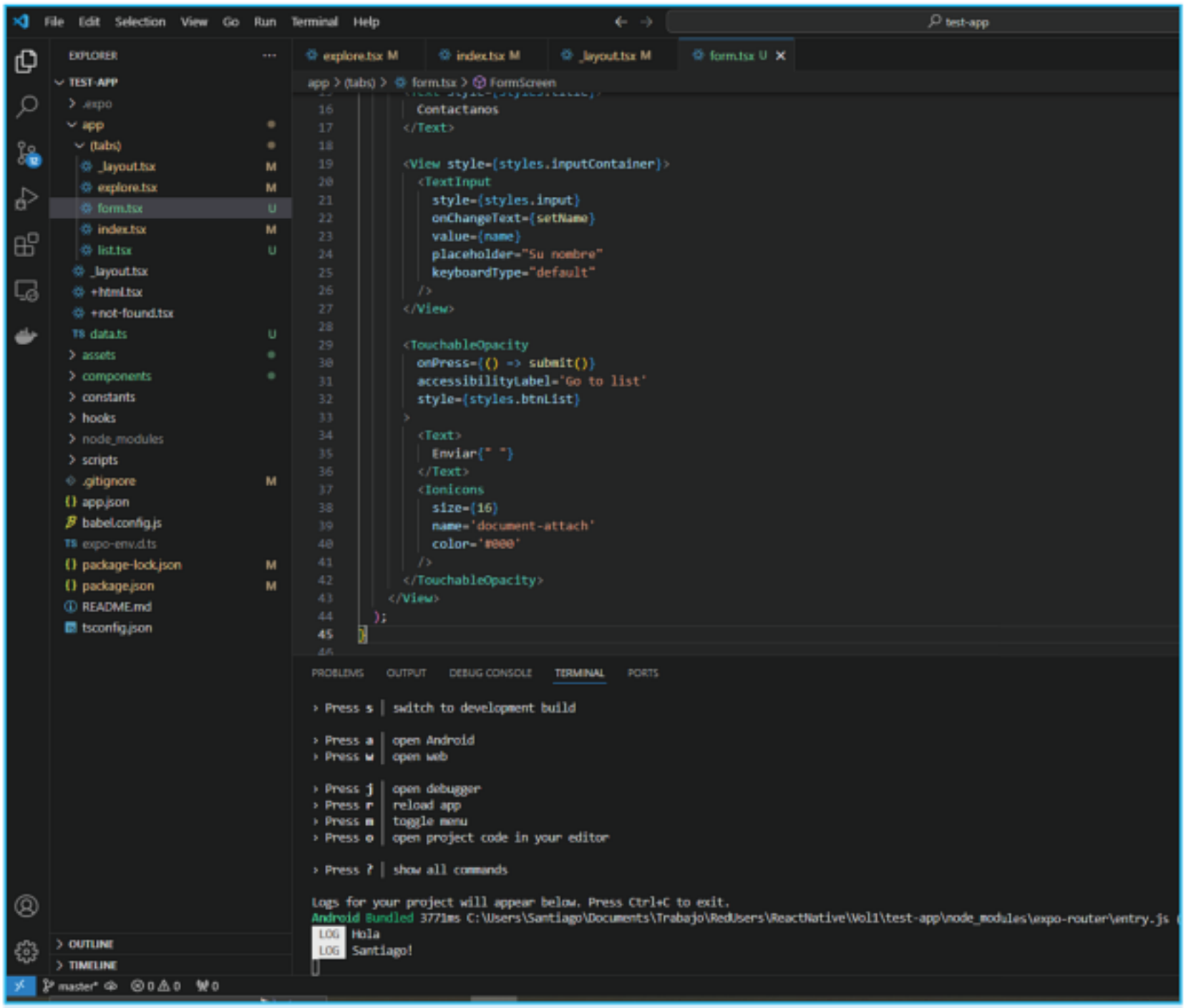


Figura 4.7. *La función console.log muestra en la terminal de la computadora el resultado que le envíes como parámetro.*

Ahora que ya sabes cómo configurar los componentes de React Native, puedes utilizar este formulario capturando los valores y enviando la información a un endpoint o servicio web disponible para su proceso en un servidor. Si bien no creaste ningún servidor, puedes generar la petición para este formulario y luego configurarlo.

En este caso, utilizaremos **`axios`** como librería para las peticiones HTTP. Comienza por importarlo dentro de la pantalla, de esta manera:

```
import axios from ‘axios’;
```

Luego, elimina el contenido de la función **submit()** y reemplázalo por este objeto:

```
const submit = async () => {
     console.log(name);

     const options = {
          method: ‘post’,
          url: url,
          body: {
               name: name,
               message: message
          }
     };
}
```

El objeto **options** es una constante que requiere la librería **axios**, aunque puedes llamarlo de otra manera. Necesita algunas características para definir qué tipo de petición vas a realizar. En primer lugar, tienes que declarar qué método **HTTP** se va a utilizar: **GET**, **POST**, **PUT** u otro similar.

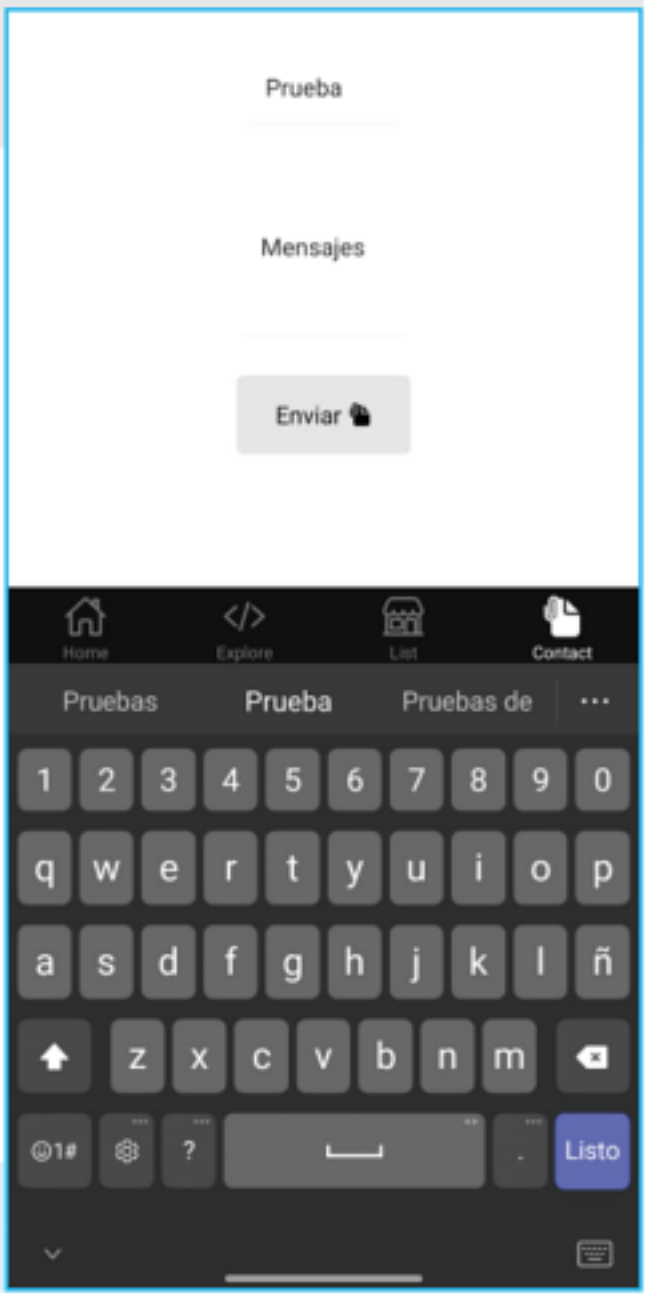


Figura 4.8. *Puedes probar tus formularios en el emulador de Expo.*

4. API externa

Por otro lado, debes declarar un endpoint de tu servicio web, es decir, la URL; y por otro, si se trata de una petición **POST** y así lo requiere el recurso, puedes declarar un **body** para la petición. Para el envío de un formulario esto es ideal, ya que puedes mandar allí la información que el usuario haya declarado.
Una vez hecho esto, solo debes realizar la petición de esta forma:

```
const url = “”;
const submit = async () => {
    console.log(name);

    const options = {
        method: ‘post’,
        url: url,
        body: {
            name: name,
            message: message
        }
    };
    const response = await axios(options);
    console.log(response.data);
}
```

Dado que no cuentas con un servicio web que te permita realizar peticiones **POST**, no verás una respuesta satisfactoria, excepto que encuentres un endpoint público con este tipo de datos esperados como cuerpo de la petición. Para estos casos, es muy útil el paquete **`json-server`**, una herramienta que permite configurar rápidamente un servidor de datos falso en tu computadora para probar una aplicación del frontend. Para esto, crea una carpeta fuera de tu proyecto de React Native, llamada **`server`**,

y ábrela con el editor Visual Studio Code. Luego, ejecuta el comando **`npm install json-server`** para instalar el paquete de **`npm`**. A continuación, solo necesitas crear un archivo **`db.json`**, con el siguiente código:

```
{
    “messages”: [
        {
            “id”: “1”,
            “name”: “John doe”,
            “message”: “Test message”
        },
        {
            “id”: “7017”,
            “name”: “name”,
            “message”: “message”
        },
        {
            “id”: “049e”,
            “name”: “name”,
            “message”: “message”
        }
    ]
}
```

Esto generará un endpoint para pruebas rápido, que verás en la terminal, llamado **`http://localhost:3000/messages`**. Este te permitirá enviar tus datos desde un formulario, por medio de una petición **`POST`**.
Utiliza este endpoint en tu proyecto de React Native de esta manera:

4. API externa

```
export default function FormScreen() {
    const url = “http://localhost:3000/messages”;

    const submit = async () => {
        const options = {
        method: ‘post’,
        url: url,
        body: {
            name: name,
            message: message
        }
    };
    const response = await axios(options);
    console.log(response);
}
```

Ahora que sabes cómo trabajar con peticiones HTTP, puedes adaptar la aplicación a tu gusto para desarrollar tu propio software que sirva a las necesidades del proyecto que estés llevando adelante.

Actividades

A continuación se presentan las preguntas y los ejercicios que deberías saber responder y resolver para considerar aprendido el capítulo.

Test de autoevaluación

1. *¿Para qué se utiliza la librería Axios?*
2. *¿Qué ventajas trae la separación de los proyectos en desarrolladores y proyectos del backend y frontend?*
3. *¿Qué propiedades conoces del componente TextInput?*
4. *¿Cómo puedes obtener los datos ingresados en un TextInput?*

Ejercicios prácticos

1. *Crea un elemento extra en tu formulario de tipo TextInput.*
2. *Agrega el campo para que permita texto plano y un límite de 50 caracteres.*
3. *Agrega una validación sobre los campos de texto.*
4. *Muestra un mensaje cuando se intente enviar el formulario sin completar todos los campos utilizando el hook de estado.*

Glosario

Arreglos: son estructuras de datos que permiten almacenar un conjunto de elementos relacionados bajo un solo nombre.

Emulador: es un software diseñado para simular que estás utilizando un dispositivo distinto, imitando por completo desde su hardware hasta sus bugs. La meta es hacer que tu sistema luzca exactamente como si fuera otro, ya sea un tipo distinto de PC, un dispositivo móvil o cualquier configuración de hardware.

Gestor de paquetes: mantiene un registro del software que está instalado en su ordenador, y le permite instalar software nuevo, actualizarlo a versiones más recientes, o eliminar software de una manera sencilla.

Ionic: es un conjunto de herramientas de interfaz de usuario de código abierto para crear aplicaciones móviles, web y de escritorio multiplataforma utilizando tecnologías web como HTML, CSS y JavaScript/TypeScript.

Kernel: es el componente central de un sistema operativo y sirve como interfaz principal entre el hardware físico de la computadora y los procesos que se ejecutan en ella a través del software. En otras palabras, podríamos definir el kernel como el corazón o núcleo del sistema operativo.

Kotlin: es un lenguaje de programación multiplataforma, estáticamente tipado, de alto nivel y propósito general con inferencia de tipos. Kotlin está diseñado para ser totalmente interoperable con Java, y la versión JVM de la biblioteca estándar de Kotlin depende de la Java Class Library, pero la inferencia de tipos permite que su sintaxis sea más concisa.

Objetive-C: es un lenguaje de programación orientado a objetos creado como un superconjunto de C para que implementase un modelo de objetos parecido al de Smalltalk.

RU

Sintaxis: hace referencia a un conjunto de reglas, principios y convenciones que rigen la estructura y el orden de las palabras y símbolos en un lenguaje de programación específico. Estas reglas definen cómo se escriben las instrucciones y cómo se organizan las diferentes partes del código fuente.

TypeScript: es un lenguaje de programación libre y de código abierto desarrollado y mantenido por Microsoft. Es un superconjunto de JavaScript, que esencialmente añade tipos estáticos y objetos basados en clases.

User experience: la experiencia del usuario es el conjunto de factores y elementos relativos a la interacción del usuario con un entorno o dispositivo concretos, dando como resultado una percepción positiva o negativa de dicho servicio, producto o dispositivo.

Valor booleano: representa un valor de verdad; es decir, TRUE o FALSE. Una expresión booleana o predicado puede producir un valor de desconocido (unknown), que está representado por el valor nulo.

Colección REACT





